강의

칼만 필터 기초와 응용

갱신식에서 베타 추정과 페어 트레이딩까지

개념 · 이론 · 방정식 · 예제 · 칼만 필터와 이동평균 비교 · 회귀 베타 · 페어 트레이딩 · VIX와 KOSPI

최적 재귀 추정기가 잡음이 섞인 관측값에서 참 상태를 복원하는 원리와,
트레이더가 이를 적응형 평균·시변 베타·헤지 비율에 활용하는 방법을 다룬다.

칼만 필터는 잡음이 섞인 센서 정보를 결합한다

칼만 필터란 무엇인가: 유래와 대표 응용

칼만 필터

- Rudolf Kalman(1930~2016)의 이름을 땄다
- 아폴로 계획에 쓰이면서 널리 알려졌다
- 대표 응용: GPS, 컴퓨터 비전, 신호 처리
- 그림: 관성측정장치(Inertial Measurement Unit, IMU)·주행거리계·GPS를 결합한 위치 추정

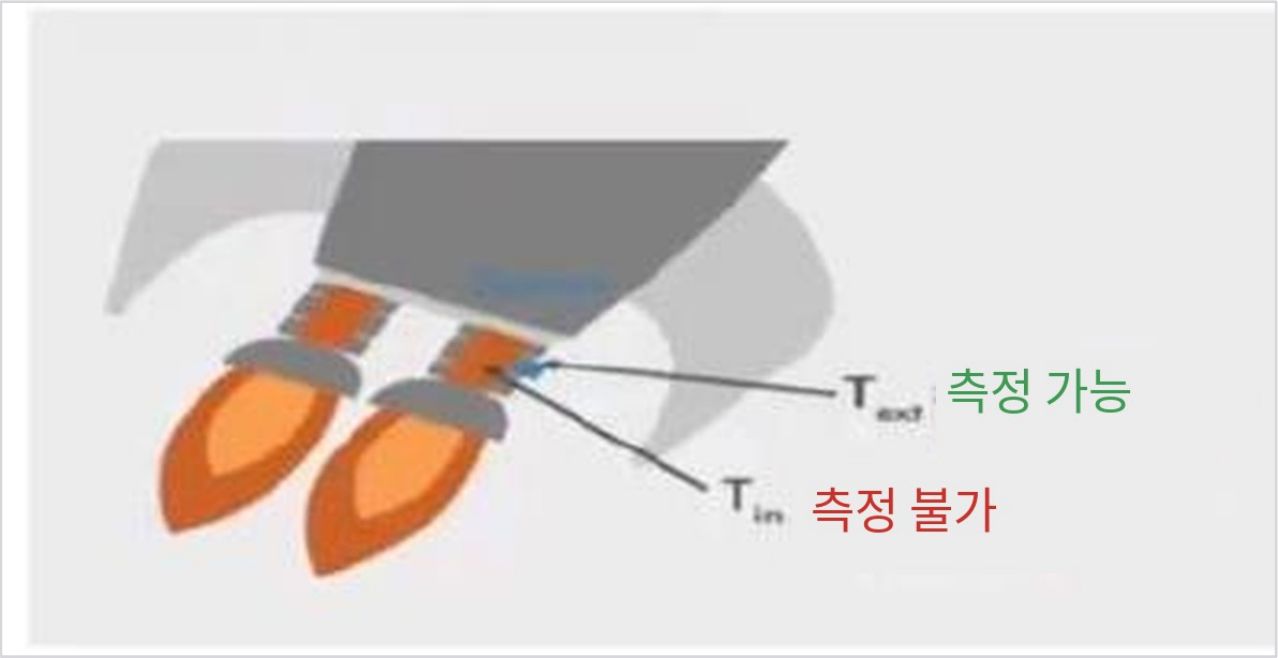


칼만 필터는 머신러닝이 아닌 최적 추정기다

칼만 필터란 무엇인가: 주요 성질

주요 성질

- 최적 추정 알고리즘이다
- 머신러닝으로 분류하지 않는다
- 과거·현재·미래 상태를 추정한다
- 시스템의 정확한 성질을 몰라도 작동한다
- 변수를 간접적으로만 측정할 수 있을 때 유용하다
- …잡음이 섞인 여러 출처에서 측정할 때도 유용하다



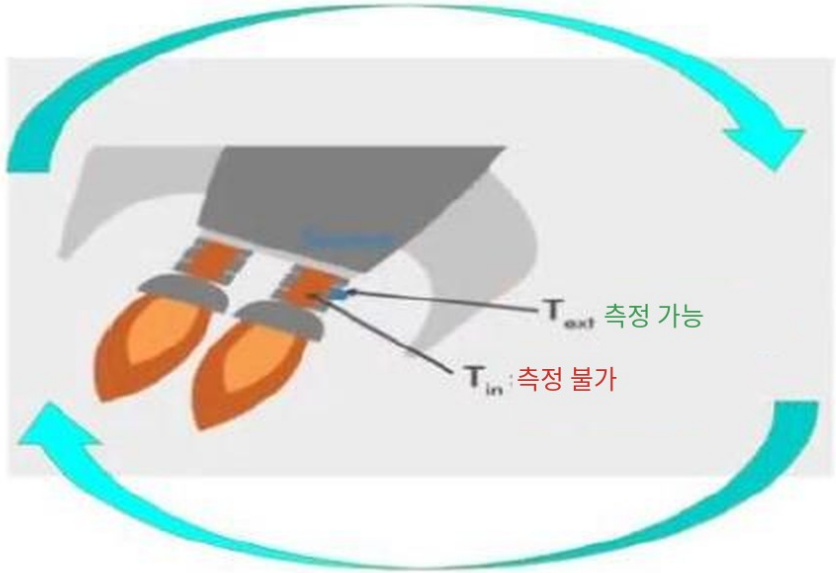
필터는 숨은 참 상태를 복원한다

칼만 필터란 무엇인가: 시간 갱신(예측)과 관측 갱신(보정)

기본 발상

- 잡음이 섞인 관측값이 순서대로 주어지면 칼만 필터(Kalman Filter, KF)는 추적 대상의 “참 상태”를 복원한다
- 반복: 시간 갱신(“예측”) ↔ 관측 갱신(“보정”)

시간 갱신
(“예측”)



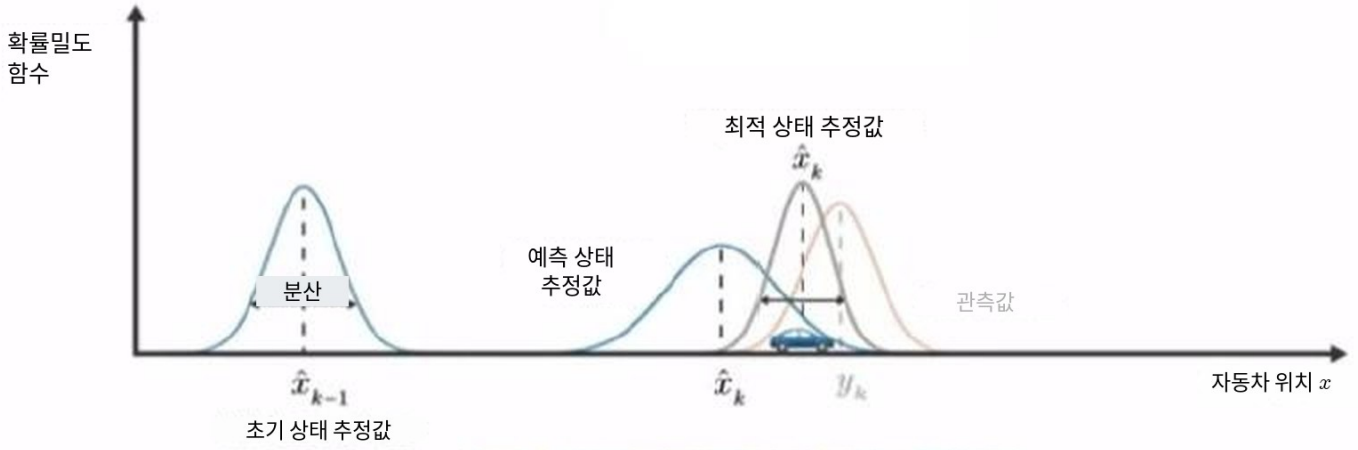
관측 갱신
(“보정”)

예측과 관측값을 하나로 결합한다

칼만 필터 이론: GPS 센서로 자동차 위치 추정

목표

- GPS 센서로 자동차의 위치를 추정한다
- 여러 시점에 걸쳐 관측한 상태 추정값을 사용한다
- 자동차 실제 위치의 최적 추정값을 구한다



Source: https://www.youtube.com/watch?v=VFXf1lIZ3p8

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k, y_{1:k-1})\, p(x_k \mid y_{1:k-1})}{p(y_k \mid y_{1:k-1})}$$

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k)\, p(x_k \mid y_{1:k-1})}{p(y_k \mid y_{1:k-1})}$$

입력 세 가지와 모형 두 개로 필터가 작동한다

칼만 필터: 목표

목표

- 대상의 “상태”를 예측한다
- 예: 자동차의 위치, 자산의 가격

이미 아는 변수 세 가지

- 이전 상태 추정값
- 모형으로 예측한 참 상태
- 참 상태의 관측값

모형 두 개

- 상태식(상태 전이 방정식)
- 관측식

새 추정값은 모형과 자료 사이에 놓인다

칼만 필터: 현재 상태, 새 상태 예측값, 새 관측값, 새 추정값



절차

매 주기마다 먼저 예측하고 그다음 보정한다

칼만 필터의 전체 절차

1. 예측 단계

이전 자료로 추정값과 그 불확실성을 계산한다.

2. 보정(갱신) 단계

추정값을 실제 관측값과 비교해 추정값을 갱신한다.

예측 → 측정 → 보정 → 반복

01

1부

칼만 필터 해설

누적 평균에서 이득과 오차 공분산(error covariance)까지

갱신값은 이전 추정값에 이득만큼의 보정을 더한 값이다

갱신의 이해: 누적 평균을 혁신(innovation) 단계로 다시 쓴다

재귀 평균

$\hat{u}_{99}$: b_1, b_2, …, b_{99}의 평균

$b_{100} \rightarrow \hat{u}_{100}$

$$\hat{u_{100}} = \frac{99}{100}\hat{u_{99}} + \frac{1}{100}b_{100}$$

$$= \hat{u_{99}} + \frac{1}{100}(b_{100} - \hat{u_{99}})$$

혁신과 이득 계수

$b_{100} - \hat{u}_{99}$: 혁신

$\frac{1}{100}$: 이득 계수

새 추정값은 이전 값에서 새 자료 쪽으로 이득이 정한 비율만큼 이동한다

칼만 필터는 α가 동적인 저역 통과 필터(low-pass filter)다

칼만 필터 해설: 평균과 비교

평균

$$\text{평균}: \quad \bar{X}_K = \frac{X_1 + X_2 + \cdots + X_K}{K} \Rightarrow \bar{X}_{K-1} = \frac{X_1 + X_2 + \cdots + X_{K-1}}{K-1}$$

$$\Rightarrow K\,\bar{X}_K = X_1 + X_2 + \cdots + X_{K-1} + X_K = (K-1)\,\bar{X}_{K-1} + X_K$$

$$\Rightarrow \bar{X}_K = \left(\frac{K-1}{K}\right)\bar{X}_{K-1} + \left(\frac{1}{K}\right)X_K$$

1차 저역 통과 필터

$$\bar{X}_K = \alpha\bar{X}_{K-1} + (1-\alpha)\,X_K$$

칼만 필터(α를 동적으로 조정)

$$\begin{aligned}\hat{X}_t &= \hat{X}_t^- + K_t(y_t - H\,\hat{X}_t^-) \\ &= (1-K_t)\,\hat{X}_t^- + K_t\,y_t \quad (H=1\text{로 가정}) \\ &= \alpha\hat{X}_t^- + (1-\alpha)\,y_t\end{aligned}$$

알고리즘은 예측, 이득, 추정, 공분산 순으로 진행된다

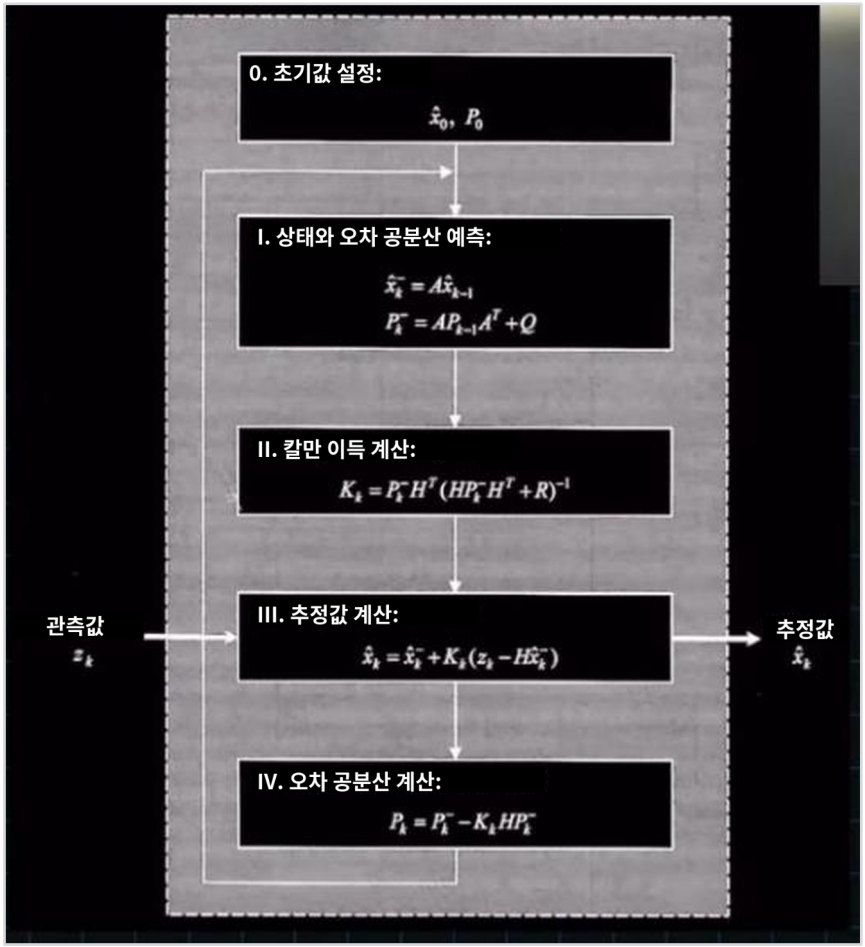
칼만 필터 해설: 칼만 필터 알고리즘

I. 예측 단계

$$X_t,\ \hat{X}_t,\ \hat{X}_t^-$$

$$X_t \sim \mathrm{N}(\hat{X}_t,\ P_t)$$

II~IV: 추정 단계



H = 1이면 갱신은 가중평균이 된다

칼만 필터 해설: II~IV 추정 단계와 칼만 이득(Kalman gain)

II~IV: 추정 단계

$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - H\hat{X}_t^-)$$
$$\hat{X}_t = \hat{X}_t^- + K_t y_t - K_t H\hat{X}_t^- = (I - K_t H)\hat{X}_t^- + K_t y_t$$

$n = 1,\ H = 1$이면:

$$\hat{X}_t = (I - K_t)\hat{X}_t^- + K_t y_t$$

$(I - K_t)$ 대신 α를 쓰면:

$\hat{X}_t = \alpha\hat{X}_t^- + (1-\alpha)y_t$: 칼만 필터

$\hat{X}_t = \alpha\hat{X}_{t-1} + (1-\alpha)X_t$: 저역 통과 필터

이득은 어떻게 구하는가?

시점 t마다 갱신되는 이득

$$K_t = P_t^- H^T(HP_t^- H^T + R)^{-1}$$

오차 공분산의 갱신

$$P_t = P_t^- - K_t H P_t^-$$

A와 Q는 예측 단계에서만 작용한다

칼만 필터 해설: I. 예측 단계 · H와 R은 추정 단계에만 나온다

$$\text{참 상태 } X_t \sim \mathrm{N}(\hat{X}_t,\ P_t)$$

I. 예측 단계

시점 t의 $\hat{X}_t$ ⇒ 시점 $t+1$의 $\hat{X}_{t+1}$

$$\hat{X}_{t+1}^{-} = A\hat{X}_t$$

$$P_{t+1}^{-} = AP_tA^T + Q$$

$\hat{X}_t$: 상태 추정값

$\hat{X}_t^{-}$: 상태 예측값

P_t : 오차 공분산 추정값

P_t^{-} : 오차 공분산 예측값

역할	기호
외부 입력	z_t (관측값)
최종 출력	$\hat{X}_t$ (추정값)
시스템 모형	$A,\ H,\ Q,\ R$
내부 계산	$\hat{X}_t^{-},\ P_t^{-},\ P_t,\ K_t$

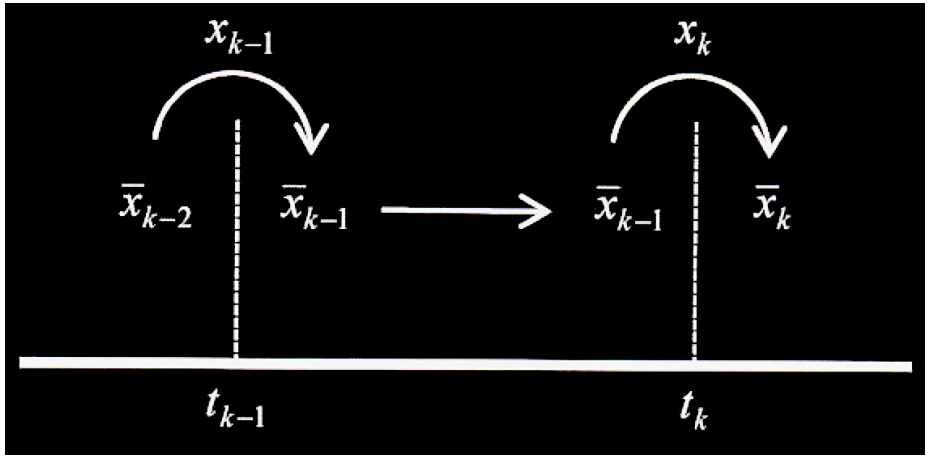
R: 관측 오차 · Q: 상태 전이 잡음

칼만 필터는 모형 기반 예측 단계를 더한다

칼만 필터 해설: 예측과 추정은 어떻게 맞물리는가?

A. 저역 통과 필터

$$\hat{X}_t = \alpha \hat{X}_{t-1} + (1-\alpha) X_t$$

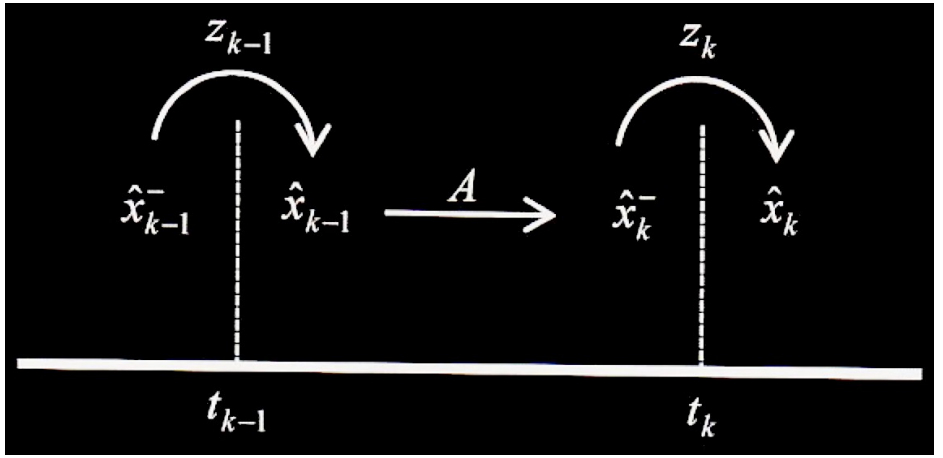


B. 칼만 필터

$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - H\,\hat{X}_t^-) \quad \Big| \quad \hat{X}_t^- = A\hat{X}_{t-1}$$

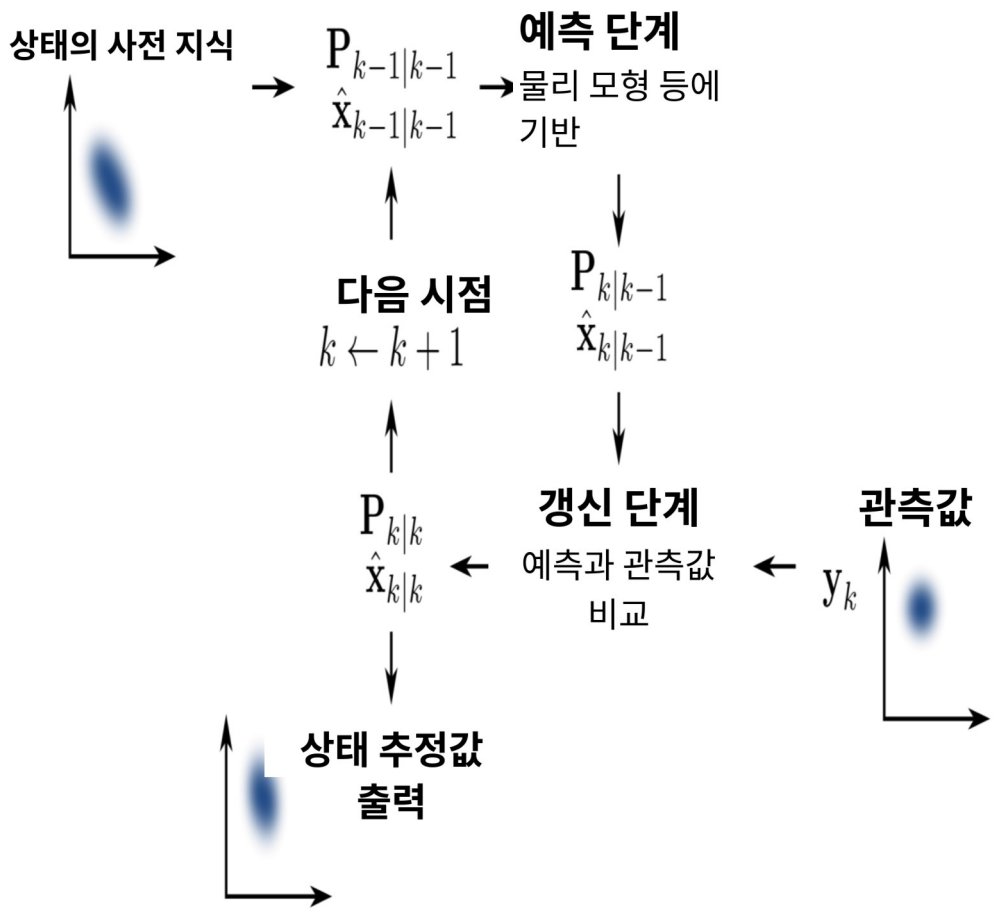
$$\hat{X}_t = \hat{X}_t^- + K_t(y_t - HA\hat{X}_{t-1})$$

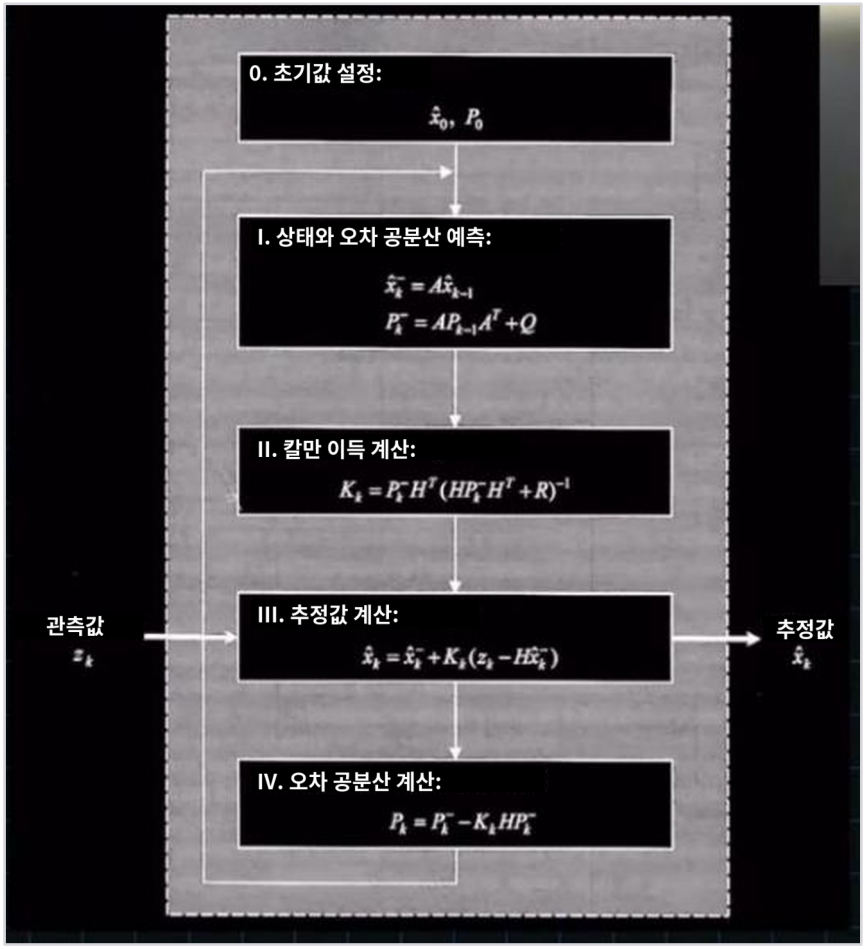
$$y \;\rightarrow\; z$$



사전 지식, 예측, 갱신, 출력을 거쳐 다시 반복한다

칼만 필터 절차 요약(왼쪽: 분포로 본 과정, 오른쪽: 알고리즘)





상태가 일정하면 R과 첫 추정값만 있으면 된다

칼만 필터 예제: 시스템 모형 파라미터 A, H, Q, R

선형 상태 모형

상태식: $X_{t+1} = AX_t + w_t$: 선형, 과정잡음
관측식: $y_t = HX_t + v_t$: 선형, 관측잡음

예제

$$X_{t+1} = X_t$$
$$y_t = X_t + v_t$$

$\Rightarrow \; X_{t+1} = AX_t, \;\; A = 1, \;\; H = 1, \;\; Q = 0, \;\; R = 4$

$\Rightarrow$ 초기 추정값 필요: $\hat{X}_0^- = 14, \;\; P_0^- = 6$

예제 2

위치와 속도로 2차원 상태 모형을 구성한다

칼만 필터 예제: 예제 2(n = 2), 위치만 관측한다

$$X_t = \begin{bmatrix} \text{위치} \\ \text{속도} \end{bmatrix}$$

$$X_{t+1} = AX_t + w_t$$

$$y_t = HX_t + v_t$$

$$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \ (2 \times 2), \qquad H = [1 \quad 0] \ \ (1 \times 2)$$

$$X_{t+1} = \begin{bmatrix} pos \\ vel \end{bmatrix}_{t+1} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} pos \\ vel \end{bmatrix}_t = \begin{bmatrix} pos_t + \Delta t * vel_t \\ vel_t \end{bmatrix}_t$$

$$y_t = HX_t + v_t = [1 \quad 0] \begin{bmatrix} pos \\ vel \end{bmatrix}_t + v_t \ \Rightarrow \ y_t = pos_t + v_t$$

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \qquad R = 10$$

초깃값

$$X_0^- = \begin{bmatrix} 0 \\ 20 \end{bmatrix}, \quad P_0^- = \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$$

02

2부

칼만 필터 요약

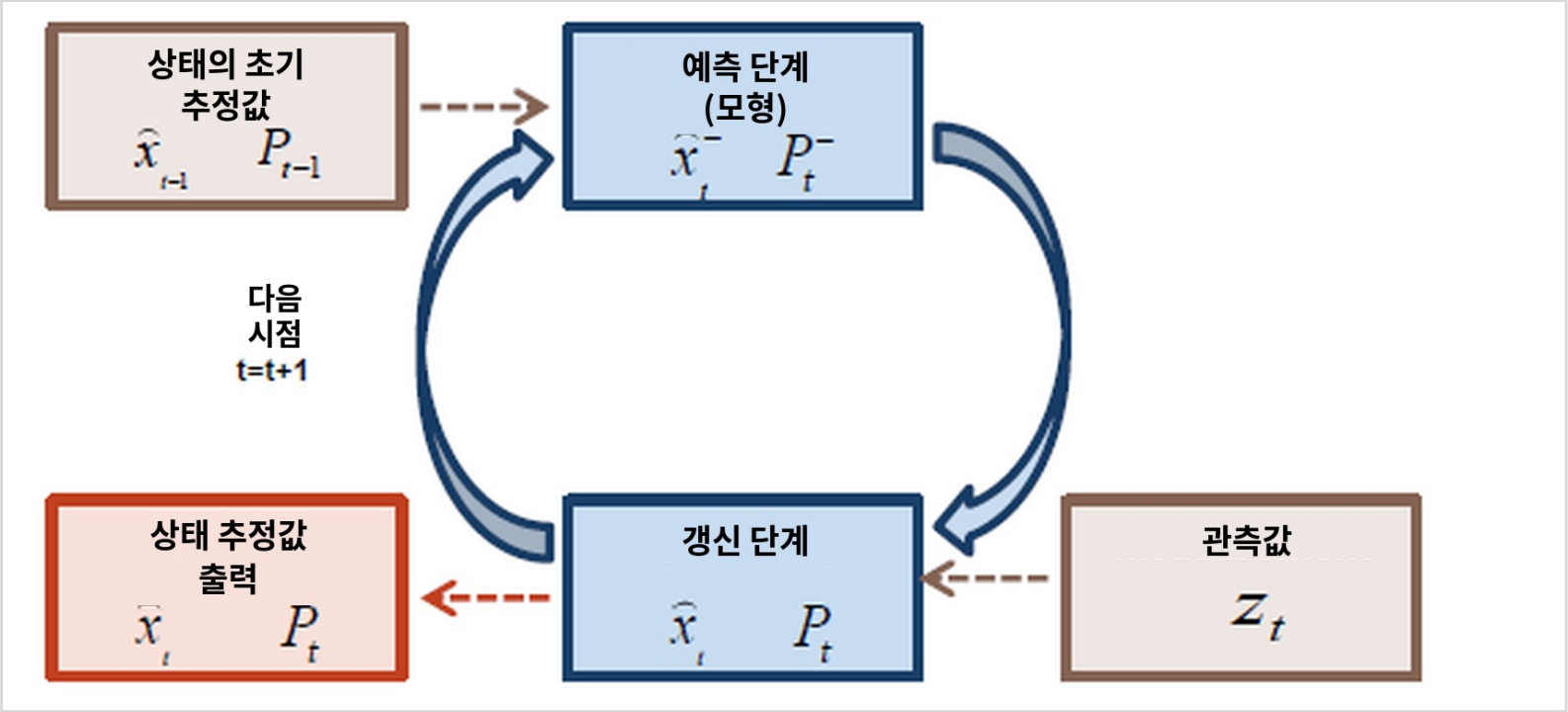
절차, 상태 갱신, 방정식, 필터 종류

추정값은 예측과 갱신의 순환을 따라 이어진다

전체 절차: 칼만 필터 순환

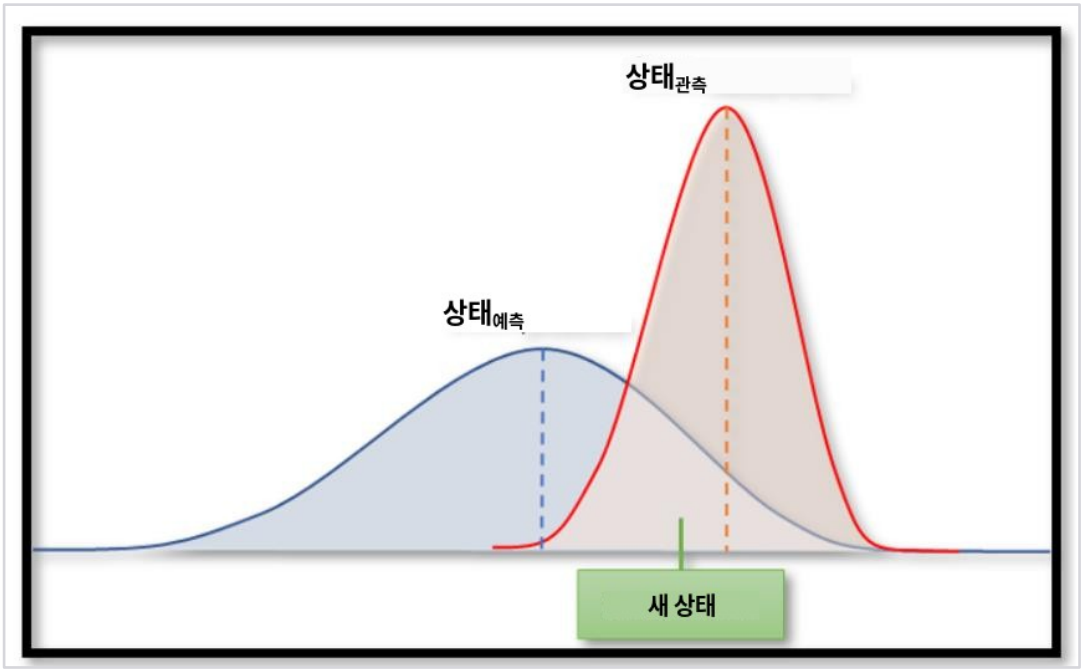
순환 과정

- 초기 추정값에서 시작한다
- 예측 단계는 모형을 사용한다
- 갱신 단계는 관측값을 사용한다
- 추정값을 출력하고 다음 시점으로 넘어간다



이득은 예측과 관측값의 가중치를 정한다

칼만 필터: 가중평균으로 본 상태 갱신(칼만 이득)



칼만 이득(0과 1 사이)

$$K_t = \frac{\text{불확실성}_{\text{예측}}}{\text{불확실성}_{\text{예측}} + \text{불확실성}_{\text{관측}}} = [0, 1]$$

상태 갱신

$$\text{상태} = K_t * \text{상태}_{\text{관측}} + (1 - K_t) * \text{상태}_{\text{예측}}, \text{ 또는:}$$
$$\text{상태} = \text{상태}_{\text{예측}} + K_t(\text{상태}_{\text{관측}} - \text{상태}_{\text{예측}})$$

상태식과 관측식으로 모형을 정의한다

칼만 방정식: 상태식, 관측식, 계수

상태식

$$x_t = Ax_{t-1} + Bu_t + w_{t-1}$$

여기서:

x_t → 예측하려는 상태 ($x \in \Re^n$)

x_{t-1} → 이전 상태

A → $n \times n$ 행렬

u_t → (선택) 제어 입력 ($u \in \Re^l$)

B → $n \times l$ 행렬

w_{t-1} → 과정잡음

관측식

$$z_t = Hx_t + v_t,$$

여기서:

z_t → 현재 관측값 $z \in \Re^m$

H → $m \times n$ 행렬

v_t → 관측잡음

계수

H : 관측 행렬(흔히 $H = I$)

$$p(w) \sim N(0, Q)$$
$$p(v) \sim N(0, R)$$

시간 갱신은 상태와 공분산을 예측한다

칼만 필터 이론: 시간 갱신(예측)과 관측 갱신의 반복

1. 시간 갱신: 상태 예측

$$\hat{x}_t = Ax_{t-1} + Bu_t$$

불확실성(공분산) 예측

$$\hat{P}_t = A * P_{t-1}A^T + Q$$

관측 갱신은 이득으로 상태를 보정한다

칼만 필터 이론: 2. 관측 갱신(이득, 상태 갱신, 공분산 갱신)

칼만 이득

$$K_t = \hat{P}_t H^T (H\hat{P}_t H^T + R)^{-1}$$

상태 갱신

$$x_t = \hat{x}_t + K_t(z_t - H\hat{x}_t)$$

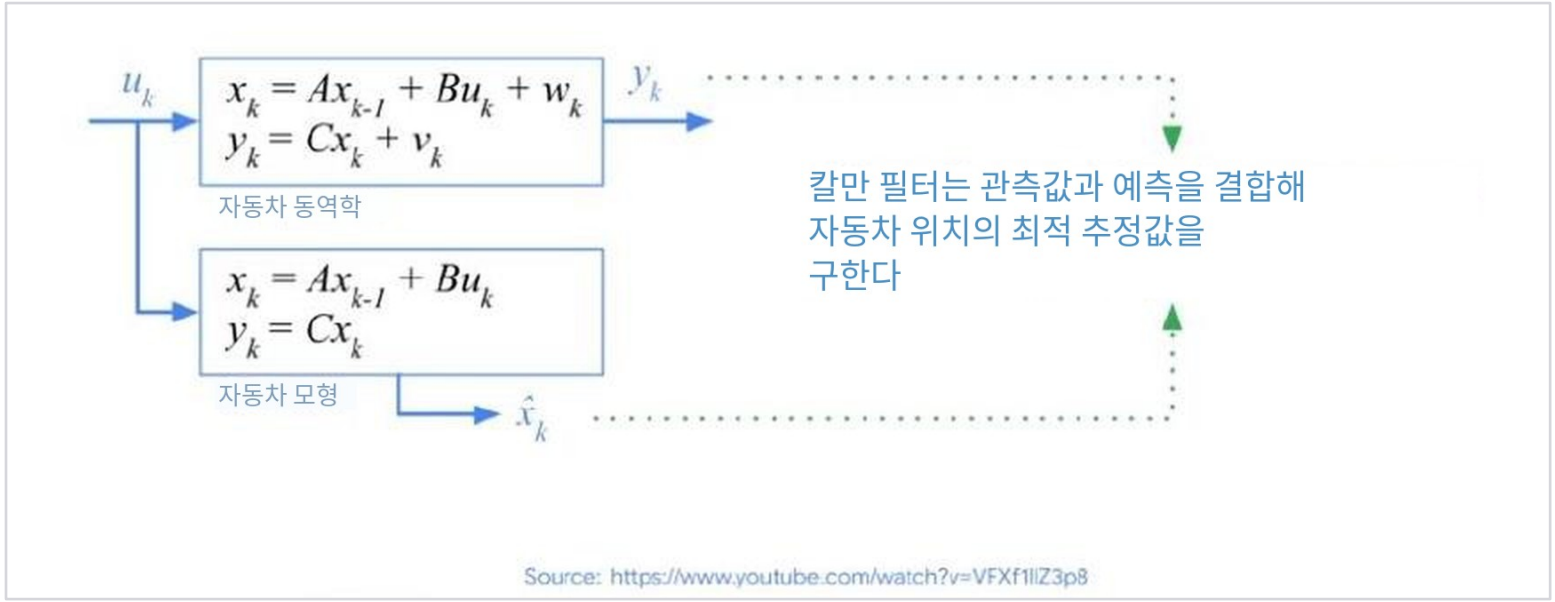
$$P_t = (I - K_t H)\hat{P}_t$$

필터는 자동차 동역학과 자동차 모형을 결합한다

칼만 필터 이론: 관측값과 예측을 결합해 자동차의 최적 위치를 구한다

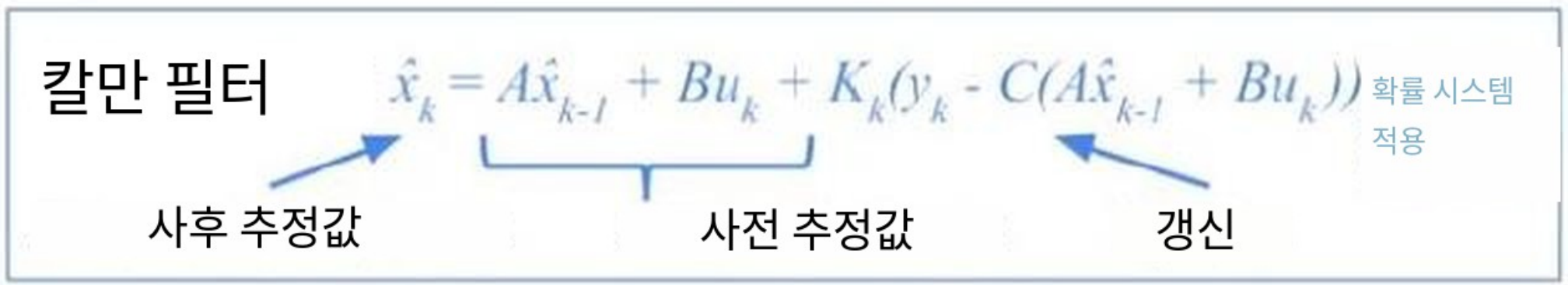
두 경로

- 자동차 동역학 → 관측값
- 자동차 모형 → 예측
- 칼만 필터는 둘을 결합해 최적 위치 추정값을 낸다



측정과 예측은 수렴할 때까지 반복된다

칼만 필터 이론: 최적 상태를 찾을 때까지 관측과 예측을 반복한다



사후 추정값(a posteriori)

결과로 얻는 갱신된 상태 추정값

사전 추정값(a priori)

관측 이전의 모형 예측값

갱신

관측값의 예상 밖 차이를 이득으로 가중해 보정한다

Source: https://www.youtube.com/watch?v=VFXf1llZ3p8

장점

재귀 갱신 덕분에 칼만 필터는 빠르고 견고하다

전체 절차: 칼만 필터 절차의 장점

결측 자료

결측 자료를 잘 처리한다

불규칙 시계열

월별·주별 자료가 섞인 것처럼 관측 간격이 불규칙한 시계열을 처리한다

빠른 반응

변화에 빠르게 반응한다

재귀 구조

처음부터 다시 계산할 필요 없이 새 자료로 모형만 갱신하면 된다

새 관측마다 적은 비용으로 한 번 갱신한다

필터는 모형·분포·계산 비용에서 서로 다르다

칼만 필터의 종류: KF, 확장 칼만 필터(EKF), 무향 칼만 필터(UKF), 입자 필터

상태 추정기	모형	가정 분포	계산 비용
칼만 필터(KF)	선형	가우시안	낮음
확장 칼만 필터(EKF)	국소 선형	가우시안	낮음(야코비안을 해석적으로 계산해야 하는 경우) 중간(야코비안을 수치적으로 계산할 수 있는 경우)
무향 칼만 필터(UKF)	비선형	가우시안	중간
입자 필터(PF)	비선형	비가우시안	높음

Source: https://www.youtube.com/watch?v=Vefia3JMeHE

03

3부

응용: 이동평균 대체

칼만 평균 추정값과 50일·100일 이동평균 비교

칼만 평균에는 윈도 길이가 필요 없다

칼만 필터와 이동평균(Moving Average, MA) 비교: AAPL, 50일·100일 이동평균

칼만 필터는...

- 시점마다 추정값을 갱신한다
- 최근 관측값에 더 큰 가중치를 둔다
- 이동평균과 달리 윈도 길이가 필요 없다

예제에서는

- 칼만 추정값과 50일·100일 이동평균을 비교한다
- 추정값은 오차항이 작은 랜덤워크(random walk)처럼 움직인다
- 모형은 다음 값 분포의 평균을 추정한다
- 초기 추정값은 평균 = 0이다

pykalman으로 가격에서 칼만 평균을 구한다

칼만 필터와 이동평균 비교: Claude Code 프롬프트

[입력] 프롬프트 · Claude Code (Code 탭)

```
kalman-lab 폴더에서 다음을 해 줘.
1. 야후 파이낸스에서 AAPL과 SPY의 수정 종가를
   2017-11-01부터 2019-11-01까지 내려받아.
   x는 AAPL로 둬.
2. pykalman으로 KalmanFilter를 만들어.
   transition_matrices=[1], observation_matrices=[1],
   initial_state_mean=0, initial_state_covariance=1,
   observation_covariance=1, transition_covariance=0.01
3. filter로 상태 평균을 구하고, x의 50일·100일
   이동평균도 계산해.
4. 칼만 추정값, x, 두 이동평균을 한 차트에 그려 줘.
```

[확인] 결과에서 볼 것

· 0에서 출발해 몇 단계 안에 가격 수준으로 뛰어오르는가?

· 이후 두 이동평균보다 AAPL을 더 가깝게 따라가는가?

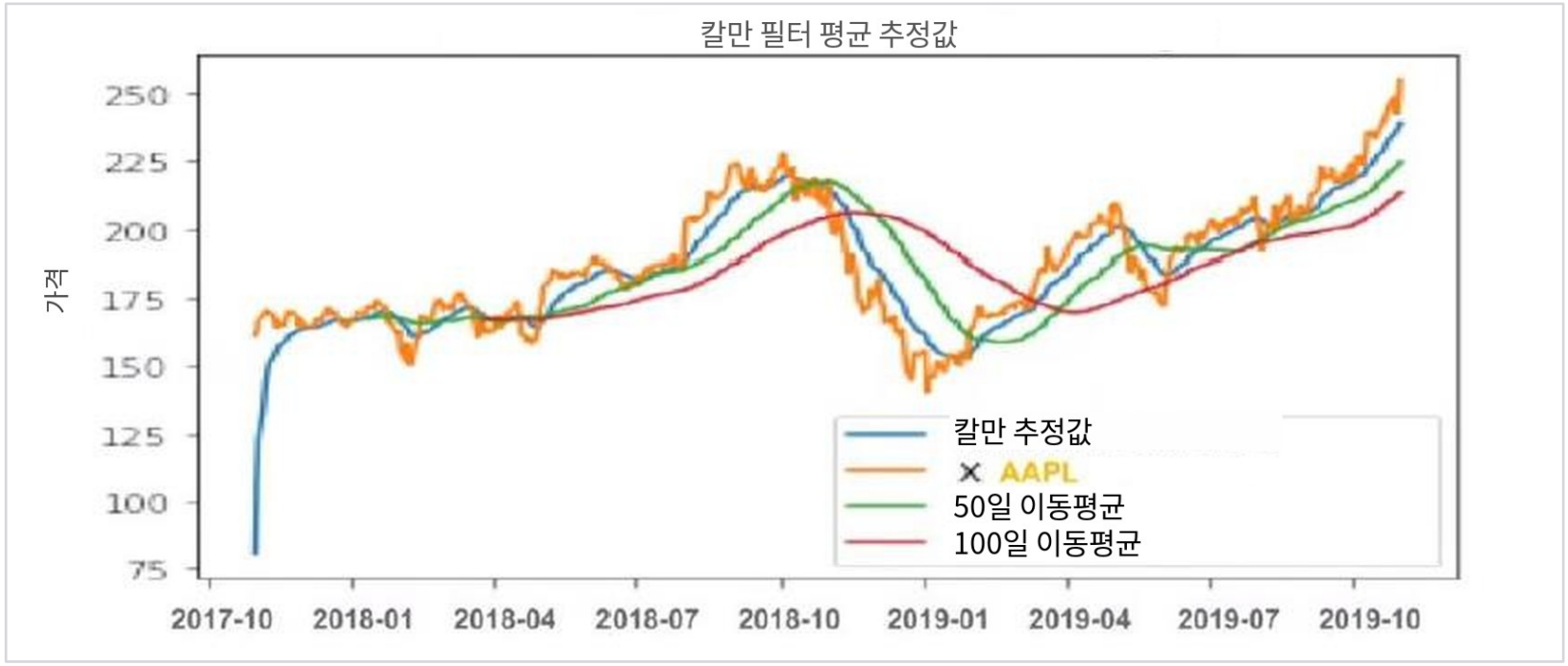
· 100일 이동평균의 반응이 가장 느린가?

칼만 추정값은 더 짧은 시차로 가격을 따라간다

칼만 추정값과 50일·100일 이동평균(AAPL, 2017~2019)

볼 점

- 0에서 출발해 몇 단계 안에 가격 수준으로 뛰어오른다
- 이후 두 이동평균보다 AAPL을 더 가깝게 따라간다

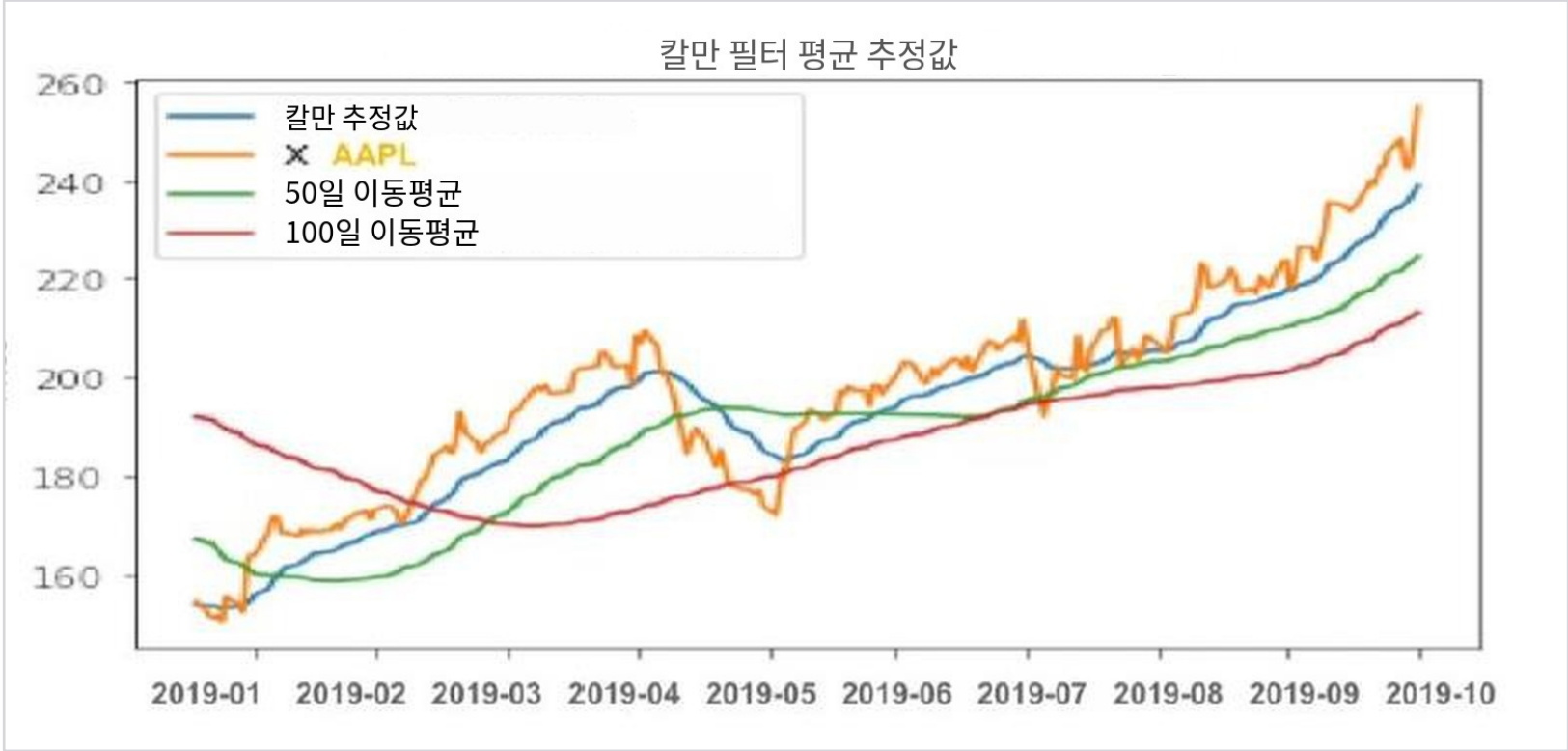


이동평균은 전환점에서 뒤처진다

칼만 추정값과 50일·100일 이동평균(AAPL, 2019)

볼 점

- 100일 이동평균의 반응이 가장 느리다
- 칼만 추정값은 가격과 50일 이동평균 사이에 놓인다



04

4부

예제: 회귀 베타 추정

SPY에 대한 AAPL의 시변 α와 β

회귀계수를 상태로 본다

칼만 필터로 회귀계수 추정: 입력

칼만 필터 입력

- 초기 시스템 상태는 관측값이 따르는 직선이다
- α와 β의 초기 추정값은 (0, 0)이다
- 새 시스템 상태는 오른쪽 식으로 주어진다

새 시스템 상태

$$(\beta, \alpha) \cdot (x_i, 1) \Rightarrow \beta\, x_i + \alpha$$

절편과 기울기는 랜덤워크를 따른다

회귀계수 추정: 관측식과 상태식

관측식

$$y_t = b_t x_t + a_t + v_t$$

y_t : 관측값(실제 값)

x_t : 독립변수(입력)

v_t : 관측잡음, $\mathcal{N}(0, R)$

상태식

$$\begin{bmatrix} a_t \\ b_t \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a_{t-1} \\ b_{t-1} \end{bmatrix} + w_t$$

a_t : 절편, 숨은 상태

b_t : 기울기, 숨은 상태

w_t : 상태잡음, $\mathcal{N}(0, Q)$

2차원 상태가 α와 β를 담고, 관측 대상은 가격이다

회귀계수 추정: α와 β를 추정하고 그리는 프롬프트

[입력] 프롬프트 · Claude Code (Code 탭)

```
같은 폴더에서 이어서 해 줘.
1. x는 SPY, y는 AAPL 가격으로 둬.
2. delta=1e-3으로 두고 전이 공분산은
   delta/(1-delta) × 2×2 단위행렬로 해.
3. 관측 행렬은 시점마다 [x, 1]로 만들어.
4. KalmanFilter를 n_dim_obs=1, n_dim_state=2로 만들어.
   상태는 (β, α) 순서, initial_state_mean=[0, 0],
   initial_state_covariance는 2×2 전부 1,
   transition_matrices는 단위행렬,
   observation_covariance=2로 해.
5. filter(y)로 순차 추정값을 구하고
   기울기와 절편을 위아래 두 패널로 그려 줘.
```

[확인] 결과에서 볼 것

· 상태의 첫 성분이 기울기(β)인가?

· 베타는 시간에 따라 크게 변하고 알파는 조금만 움직이는가?

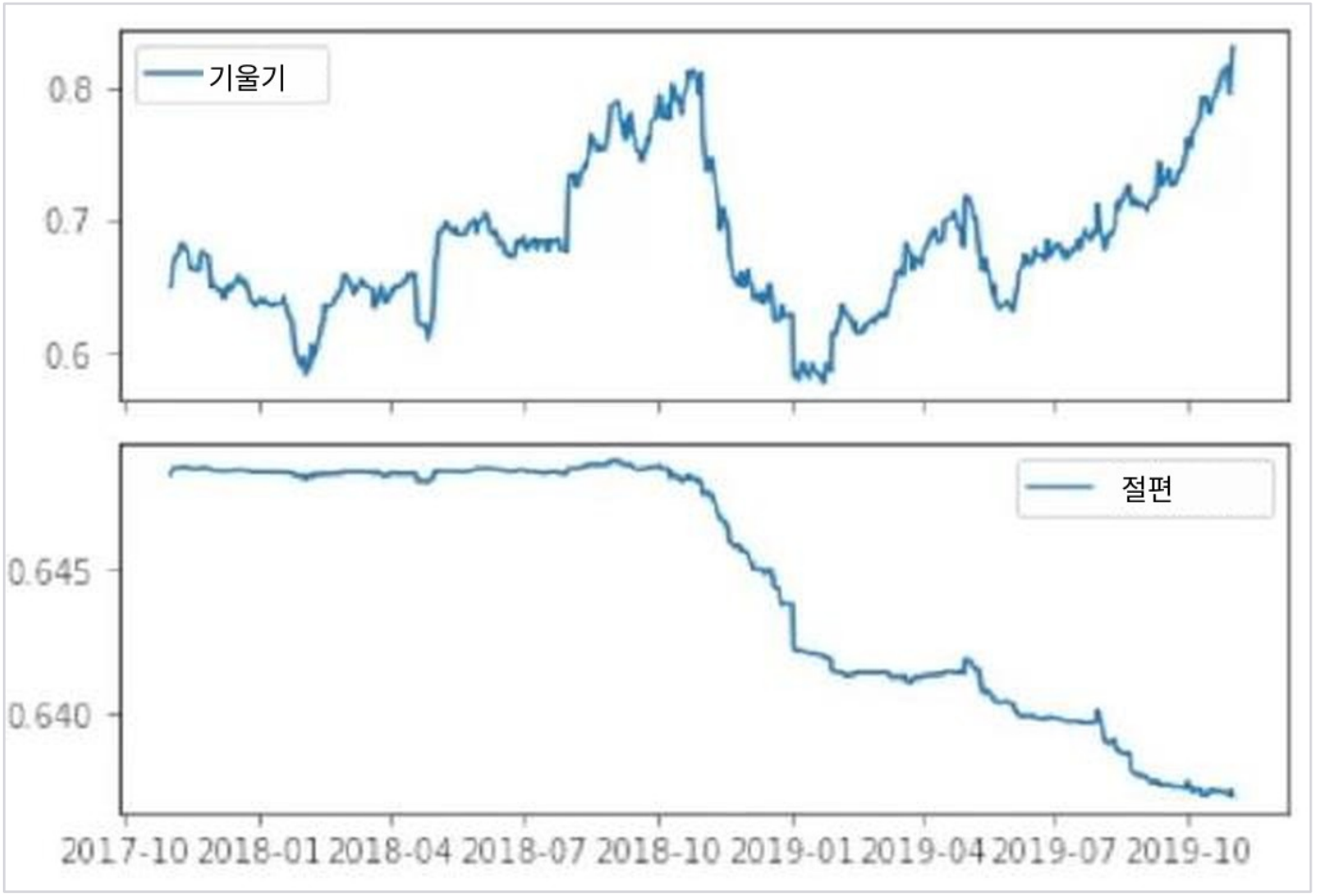
· 추정 직선이 오래된 자료보다 최근 자료에 더 잘 맞는가?

베타는 시간에 따라 크게 변하지만 알파는 거의 움직이지 않는다

SPY에 대한 AAPL의 회귀계수

시사점

- 베타는 시간에 따라 크게 변하고 알파는 조금만 움직인다
- 베타로 거래 전략을 헤지한다면 현재의 정확한 베타를 써야 한다

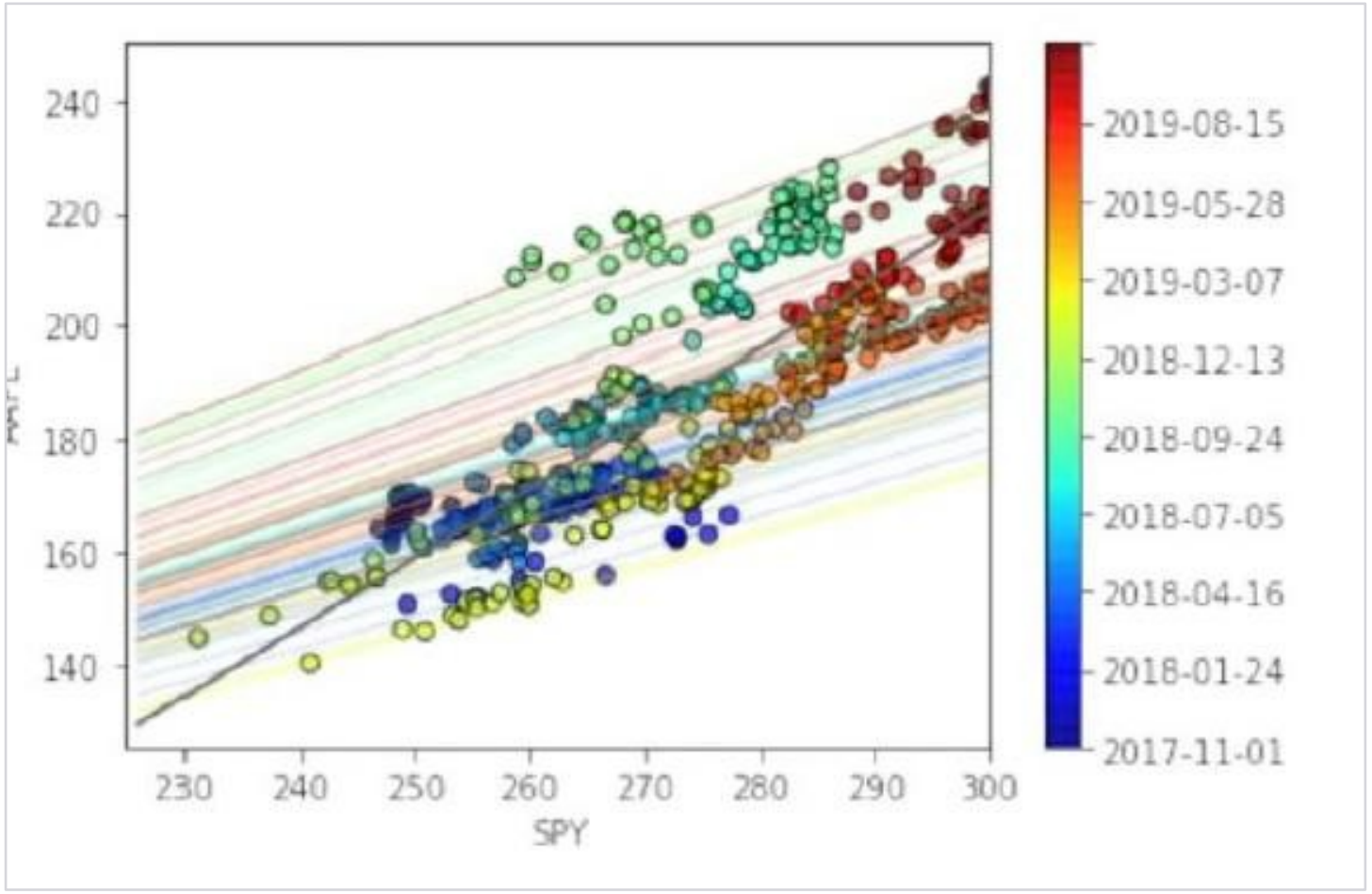


추정 직선마다 최근 자료의 영향이 가장 크다

SPY에 대한 AAPL의 회귀계수: 날짜별로 색을 입힌 직선

차트 읽기

- 각 추정값은 과거 관측값을 모두 사용한다
- ...그러나 오래된 자료보다 최근 자료에 더 잘 맞는다
- 그래서 필터는 구조 변화에 적응한다



같은 필터를 수익률에도 적용한다

AAPL·SPY 수익률로 베타 차트를 그리는 프롬프트

[입력] 프롬프트 · Claude Code (Code 탭)

```
같은 폴더에서 이어서 해 줘.
1. x, y를 pct_change로 수익률로 바꾸고 첫 행은 버려.
2. delta=1e-2로 두고 전이 공분산은
   delta/(1-delta) × 2×2 단위행렬로 해.
3. 관측 행렬은 시점마다 [x 수익률, 1]로 만들어.
4. 앞과 같은 2차원 KalmanFilter를 쓰되
   observation_covariance=0.01로 해.
5. filter(y 수익률)로 추정하고 기울기와 절편을
   두 패널로 그려 줘.
6. 수익률 산점도에 시점별 추정 직선을 날짜 색으로
   그리고, OLS 직선을 검은 선으로 겹쳐 줘.
```

[확인] 결과에서 볼 것

· 수익률이 덜 흩어져 있는가?

· 칼만 기울기가 0 근처에서 출발하는가?

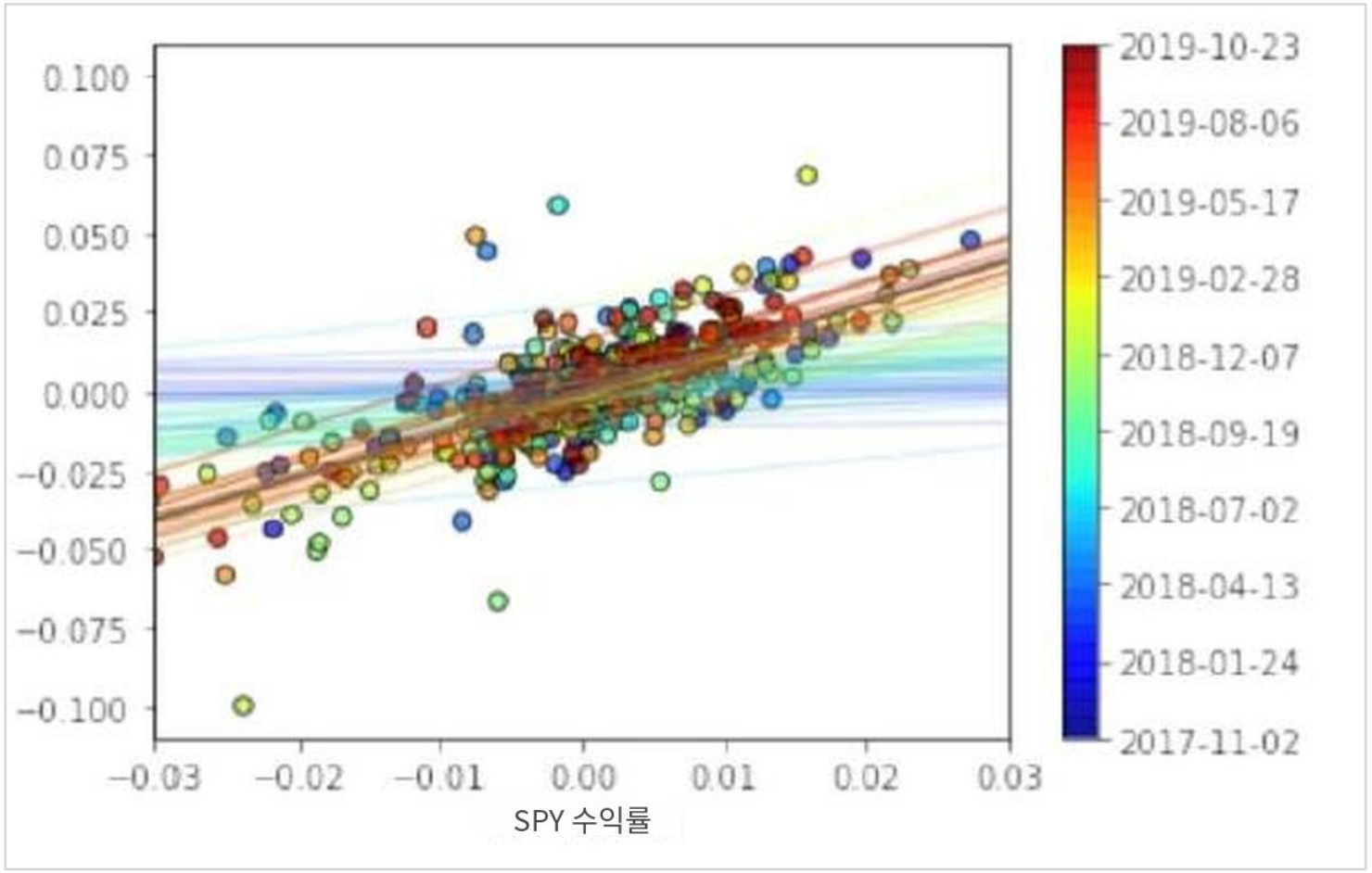
· 시간이 지나면서 높아져 OLS 계수(검은 선)와 같아지는가?

수익률에서는 기울기가 OLS 계수로 수렴한다

SPY 수익률에 대한 AAPL 수익률의 회귀계수

차트 읽기

- 수익률은 덜 흩어져 있다
- 칼만 기울기는 0 근처에서 출발한다
- 시간이 지나면서 높아져 보통최소제곱(Ordinary Least Squares, OLS) 계수(검은 선)와 같아진다



05

5부

예제: 페어 트레이딩(pairs trading)

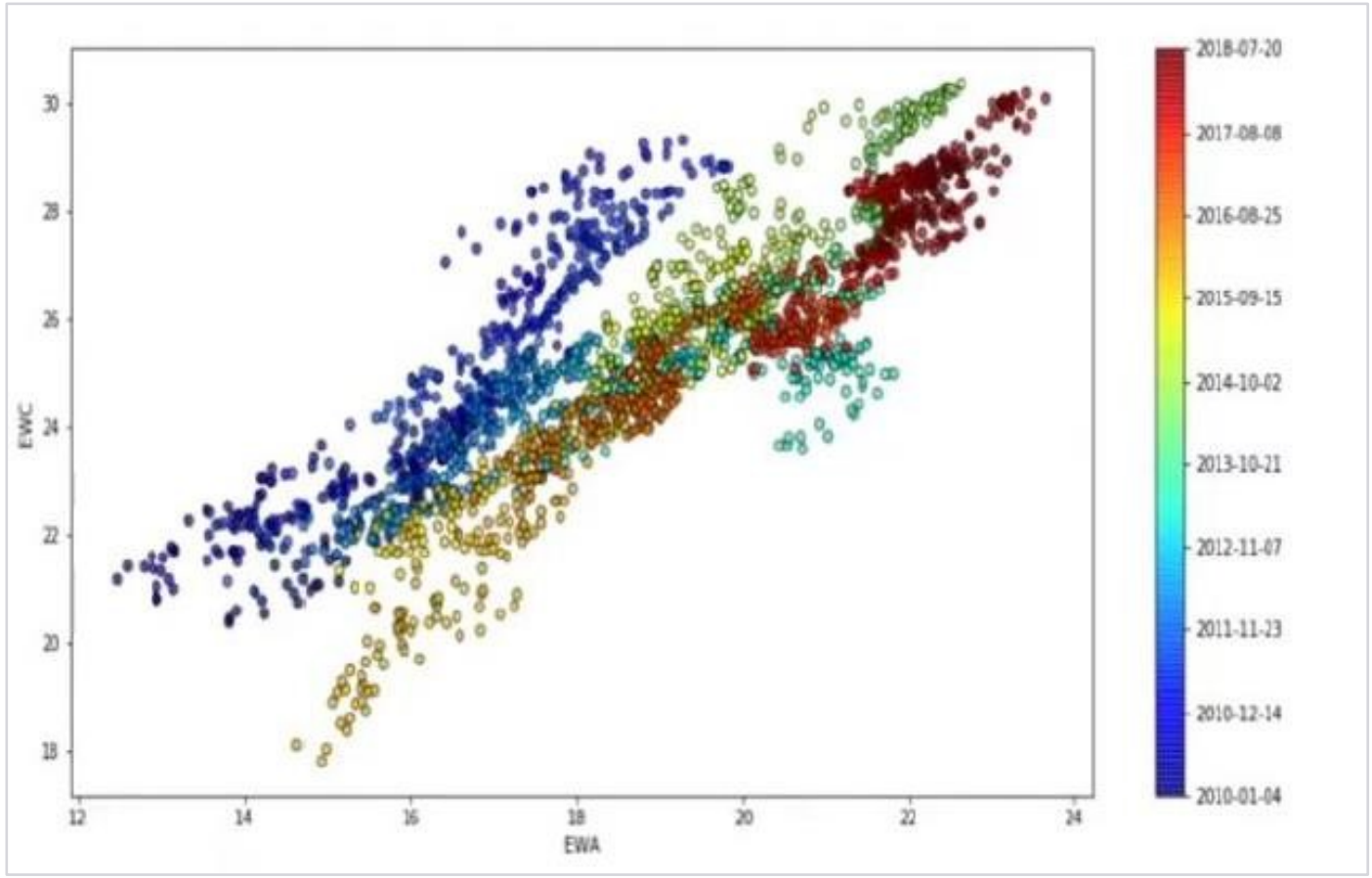
EWA 대 EWC: Ernest Chan의 동적 헤지 비율

상관이 높은 두 ETF: 한쪽으로 다른 쪽을 예측할 수 있는가?

칼만 필터 예제: 종목 상관관계 예측

질문

- 상장지수펀드(Exchange-Traded Fund, ETF)인 EWA와 EWC의 상관이 높다고 가정한다
- 한쪽을 알면 다른 쪽 가격을 예측할 수 있는가?
- 색 = 날짜



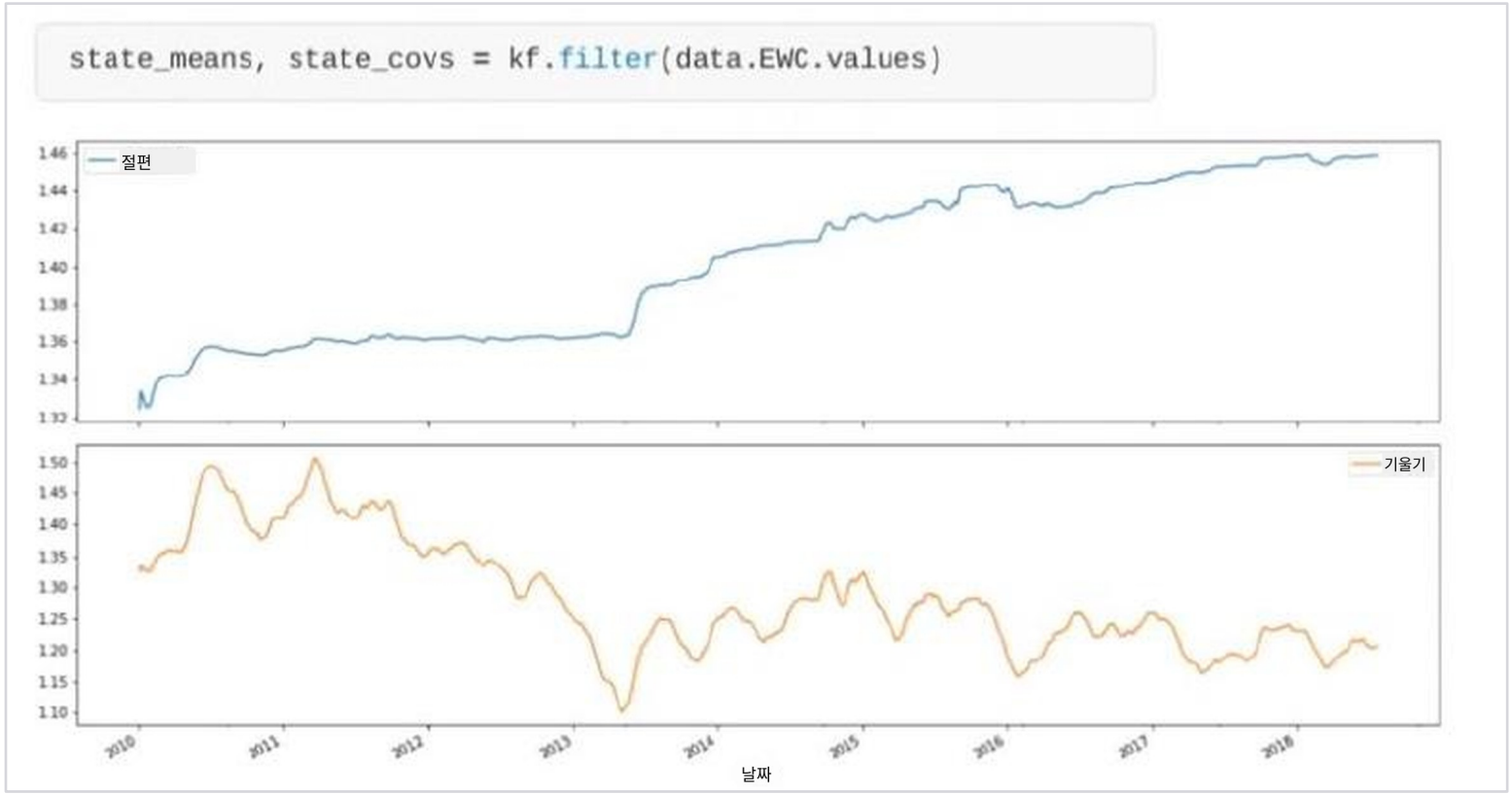
* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

EWC를 필터링하면 평균과 공분산 경로를 얻는다

칼만 필터 예제: EWC에 대한 예측 상태의 평균과 공분산

볼 점

- state_means, state_covs = kf.filter(EWC)
- 위: 절편
- 아래: 기울기

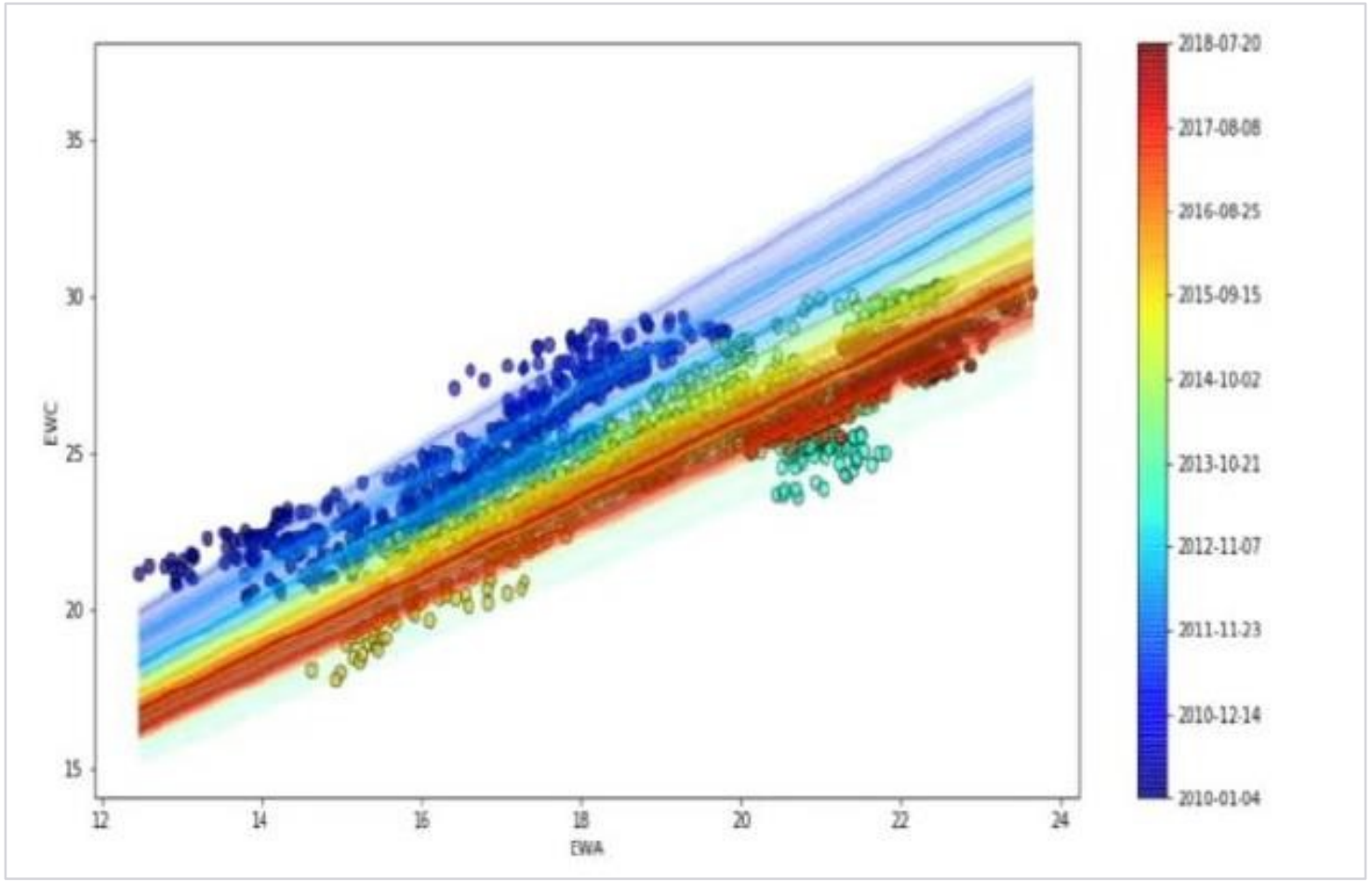


예측 직선은 시간에 따라 이동한다

칼만 필터 예제: EWC로 EWA 예측

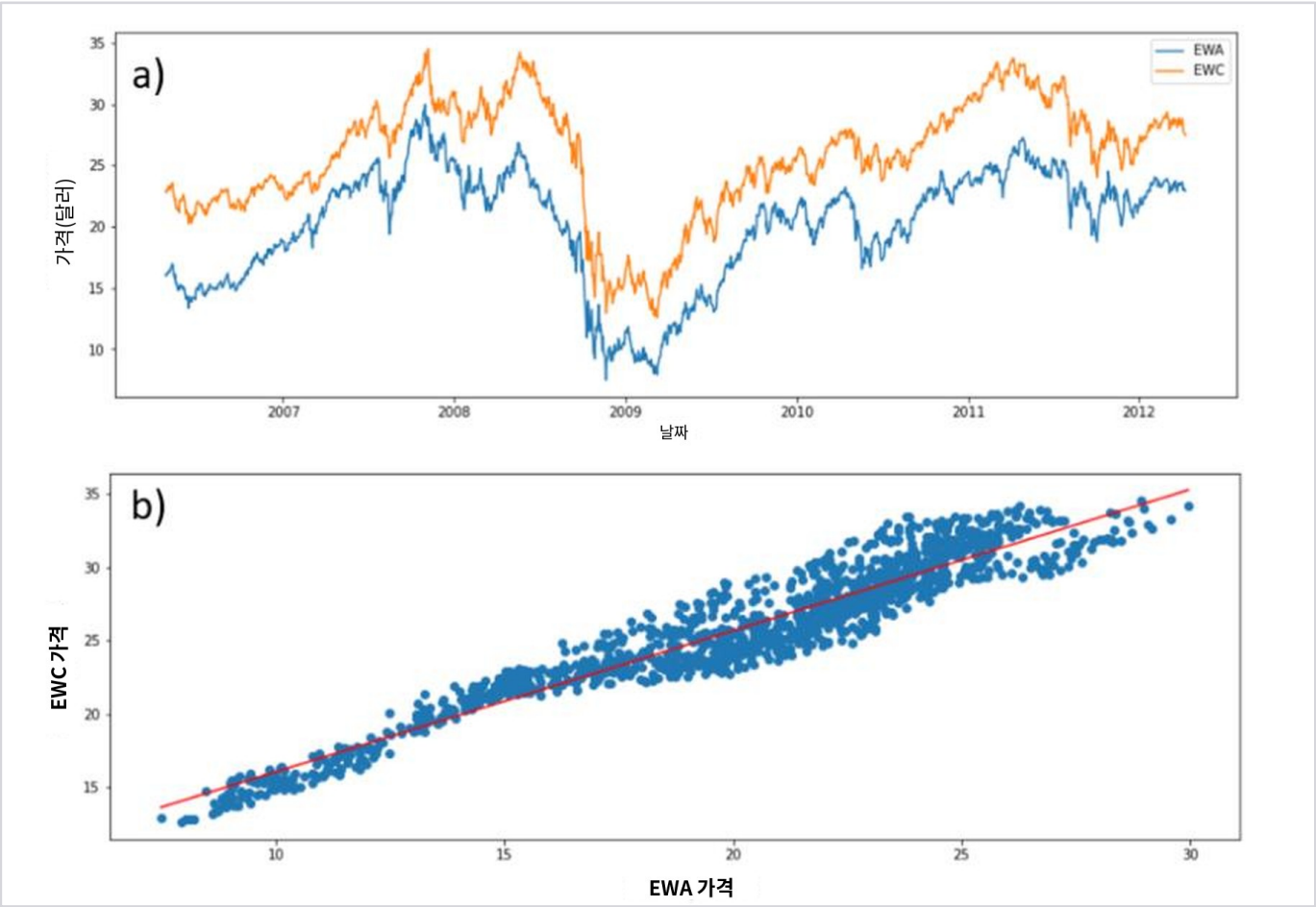
볼 점

- 상관관계 예측의 회귀선이 시간에 따라 변한다
- 색 = 날짜



고정된 OLS 헤지 비율로 EWC와 EWA를 연결한다

Ernest Chan의 페어 트레이딩 예제: EWA 대 EWC



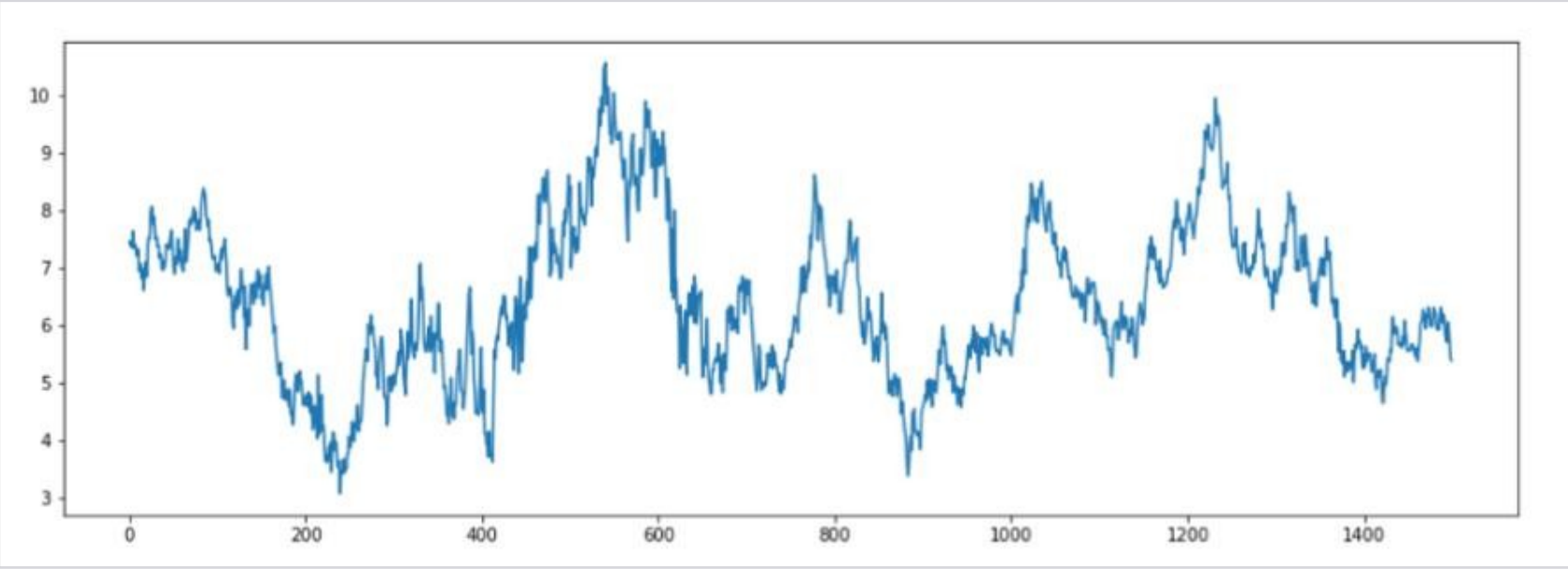
고정 헤지 비율

$$EWC = \beta EWA + \alpha$$

$$EWC - 0.9624EWA = 6.4113$$

고정 비율 스프레드가 거래 신호다

Ernest Chan의 페어 트레이딩 예제: 스프레드



$$EWC = \beta EWA + \alpha$$

$$EWC - 0.9624EWA = 6.4113$$

헤지 비율을 랜덤워크 상태로 둔다

칼만 필터 페어 트레이딩: 상태식과 관측식

상태식

$$x_t = Ax_{t-1} + w_t, \quad A = I_2$$

$$\begin{pmatrix} \beta_t \\ \alpha_t \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \beta_{t-1} \\ \alpha_{t-1} \end{pmatrix} + w_t$$

상태 벡터

$$x_t = \begin{pmatrix} \beta_t \\ \alpha_t \end{pmatrix}$$

관측식

$$z = Hx + v, \quad x = (\beta, \alpha)^T$$
$$H = (EWA, 1)$$

풀어 쓰면

$$EWC = EWA \cdot \beta + \alpha + v$$

가격은 관측값이고 헤지 비율은 상태다

칼만 필터 페어 트레이딩: 고정 헤지 비율 가정 완화

1. 상태 예측

$$A = I \text{ (단위행렬)}$$

$$\hat{x}_t = x_{t-1}$$

2. 불확실성 예측

$$\hat{P}_t = P_{t-1} + Q$$

3. 칼만 이득

$$H = \begin{pmatrix} EWA \\ 1 \end{pmatrix}^T$$

$$K_t = \hat{P}_t H^T (H * \hat{P}_t * H^T + R)^{-1}$$

참고

$$z = Hx + v, \qquad x = \begin{pmatrix} \beta \\ \alpha \end{pmatrix}$$

Chan의 설정으로 과정잡음과 관측잡음의 크기를 정한다

과정잡음 Q(2×2)와 관측잡음 R(1×1), Chan이 제안한 값으로 초기화

$$Q = \frac{\delta}{1-\delta} I, \text{ 단 } \delta = 0.0001$$

$$R = 0.001$$

δ가 작으면 헤지 비율이 천천히 변한다.
R은 가격 관측의 잡음이다

pykalman의 KalmanFilter는 이 모형에 그대로 대응한다

pykalman 모듈의 KalmanFilter 클래스로 인스턴스를 만든다

```
kf = KalmanFilter(n_dim_obs=1, n_dim_state=2,
                  initial_state_mean=np.zeros(2),
                  initial_state_covariance=np.ones((2, 2)),
                  transition_matrices=np.eye(2),
                  observation_matrices=obs_mat,
                  observation_covariance=1.0,
                  transition_covariance=trans_cov)
```

$$
\begin{aligned}
x_0 &\sim \text{Gaussian}(\mu_0, \Sigma_0) \\
x_{t+1} &= A_t x_t + b_t + \epsilon_{t+1}^1 \\
y_t &= C_t x_t + d_t + \epsilon_t^2 \\
\epsilon_t^1 &\sim \text{Gaussian}(0, Q) \\
\epsilon_t^2 &\sim \text{Gaussian}(0, R)
\end{aligned}
$$

매개변수 이름	기호
initial_state_mean	μ_0
initial_state_covariance	Σ_0
transition_matrices	A
transition_offsets	b
transition_covariance	Q
observation_matrices	C
observation_offsets	d
observation_covariance	R

클래스에 R, Q, 상태, 공분산을 저장한다

페어 트레이딩: 파이썬 구현(1/3)

```python
class MyKalmanFilter:


    def __init__(self, delta=1e-4, R=1e-3):
        # measurement noise variance
        self.R = R


        # co-variance of process noise(2 dimensions)
        self.Q = delta / (1-delta) * np.eye(2)
        # state (slope, intercept) will be (2 x n),
        # we will initialize with just one column at first
        self.x = np.zeros((2, 1))


        # state covariance
        self.P = np.zeros((2,2))
```

코드 읽는 법

`class MyKalmanFilter`	필터에 필요한 값과 계산을 한 묶음으로 정의한다.
`def __init__`	필터를 만들 때 한 번 실행된다. delta와 R을 주지 않으면 = 뒤의 값을 쓴다.
`self.`	이 필터 객체가 기억하는 값이다. 다음 시점 계산에 다시 쓴다.
`self.R = R`	관측잡음의 분산 R을 저장한다.
`self.Q = ...`	과정잡음 Q = δ/(1−δ) × 단위행렬(50쪽). np.eye(2)가 2×2 단위행렬이다.
`self.x = ...`	상태(기울기, 절편). np.zeros((2, 1))은 0으로 채운 2×1 열이다.
`self.P = ...`	상태의 공분산. 0으로 채운 2×2 행렬에서 시작한다.

step_forward는 시간 갱신과 이득을 계산한다

페어 트레이딩: 파이썬 구현(2/3)

```python
def step_forward(self, y1, y2):
    # Before entering the equations, let's define H as (1, 2) matrix
    H = np.array([y2, 1])[None]
    # and define z
    z = y1

    ## TIME UPDATE ##
    # first thing is to predict new state as the previous one (2x1)
    x_hat = self.x[:, -1][..., None]

    # then, the uncertainty or covariance prediction
    P_hat = self.P + self.Q

    ## MEASUREMENT UPDATE ##
    # calc the Kalman gain
    K = P_hat.dot(H.T)/(H.dot(P_hat.dot(H.T))+self.R)
```

코드 읽는 법

step_forward(self, y1, y2)

하루치 계산이다. y1은 EWC 가격, y2는 EWA 가격이다.

H = ... $H = (EWA,\ 1)$

[None]을 붙여 1×2 행렬로 만든다.

z = y1

오늘의 관측값이다.

x_hat = ... $\hat{x}_t = x_{t-1}$

self.x의 마지막 열(직전 상태)을 꺼내 2×1 열로 둔다.

P_hat = ... $\hat{P}_t = P_{t-1} + Q$

직전 self.P에 Q를 더한다. 불확실성이 커진다.

K = ... $K_t = \hat{P}_t H^T (H \hat{P}_t H^T + R)^{-1}$

.dot은 행렬 곱, .T는 전치다. 분모가 1×1이어서 /로 나눈다.

* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

관측 갱신은 x와 P를 보정한다

페어 트레이딩: 파이썬 구현(3/3)

```
# state update part 1 (measurement estimation)
z_hat = H.dot(x_hat)
# state update part 2
x = x_hat + K.dot(z-z_hat)

# uncertainty update
self.P = (np.eye(2)-K.dot(H)).dot(P_hat)

# append the new state to the vector
self.x = np.concatenate([self.x, x], axis=1)

return x, self.P, K, z_hat
```

왜 self.P인가

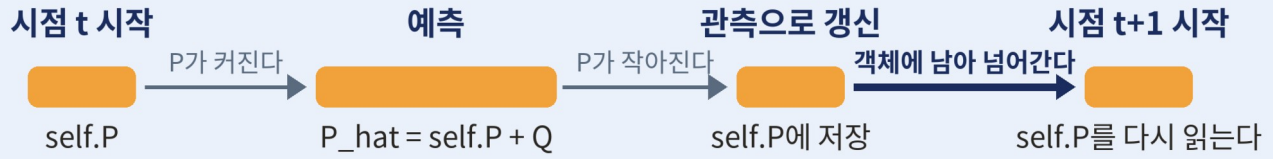
① P는 내 추정을 얼마나 믿을 수 있는지 나타내는 값이다(불확실성).

② 칼만 필터는 한 번 계산하고 끝나지 않는다. 이번에 구한 P를 다음 시점에 다시 쓴다.

③ 그래서 함수가 끝나도 사라지지 않게 필터 객체 안에 저장한다. 그 저장 칸의 이름이 self.P다.

④ self.가 없는 K·z_hat은 이번 계산에서만 쓴다. 객체에 남지 않고 다음 시점에 새로 계산한다.

x도 지역 변수지만 같은 값을 self.x에 한 열씩 쌓는다. self.P는 최신 값 하나로 덮어쓴다.



막대 길이 = 불확실성 P

* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

step_forward는 네 값을 돌려준다

return x, self.P, K, z_hat: 값의 뜻, 식, 그 값을 만든 코드 줄

반환값	뜻	식	코드 줄
x	이번에 갱신한 상태 추정값	$x_t = \hat{x}_t + K_t(z_t - H\hat{x}_t)$	`x = x_hat + K.dot(z-z_hat)`
self.P	갱신한 추정의 불확실성	$P_t = (I - K_t H)\hat{P}_t$	`self.P = (np.eye(2)-K.dot(H)).dot(P_hat)`
K	칼만 이득	$K_t = \hat{P}_t H^T (H\hat{P}_t H^T + R)^{-1}$	`K = P_hat.dot(H.T)/(H.dot(P_hat.dot(H.T))+self.R)`
z_hat	예측한 관측값	$\hat{z}_t = H\hat{x}_t$	`z_hat = H.dot(x_hat)`

self.x = np.concatenate([self.x, x], axis=1): 새 상태를 self.x의 오른쪽에 한 열 덧붙인다. 다음 시점의 x_hat이 이 마지막 열을 꺼낸다.
np.eye(2)는 식의 I, .dot은 행렬 곱이다. 다음 장의 스프레드는 돌려받은 z_hat을 쓴다.

행을 순회하면 스프레드와 헤지 비율을 얻는다

페어 트레이딩: 구현한 클래스를 돌리는 프롬프트

[입력] 프롬프트 · Claude Code (Code 탭)

```
같은 폴더에서 이어서 해 줘.
1. 앞 장의 MyKalmanFilter 클래스를 그대로 써.
2. EWA와 EWC의 일별 종가를 2006~2012년으로
   내려받아 df의 두 열로 둬.
3. MyKalmanFilter(delta=1e-4, R=1e-3)를 만들어.
4. 날짜마다 step_forward에 EWC, EWA를 넘기고
   EWC에서 예측값 z_hat을 뺀 스프레드를 모아.
5. 앞 10개를 뺀 스프레드를 그리고,
   시간에 따른 헤지 비율도 그려 줘.
```

[확인] 결과에서 볼 것

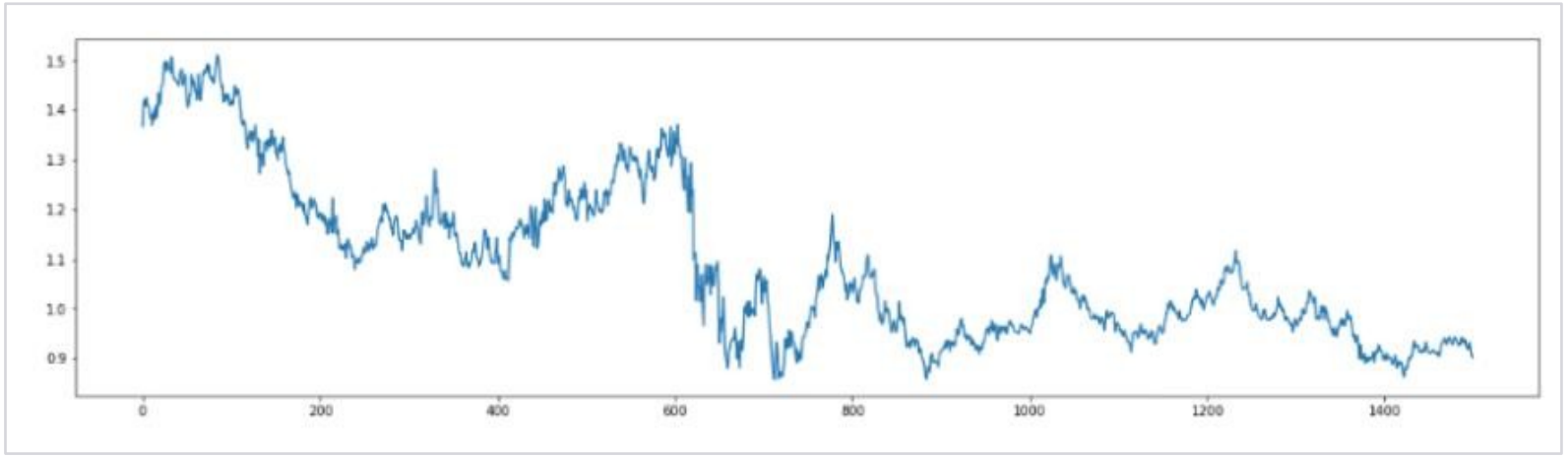
· 헤지 비율이 시간에 따라 변하는가?

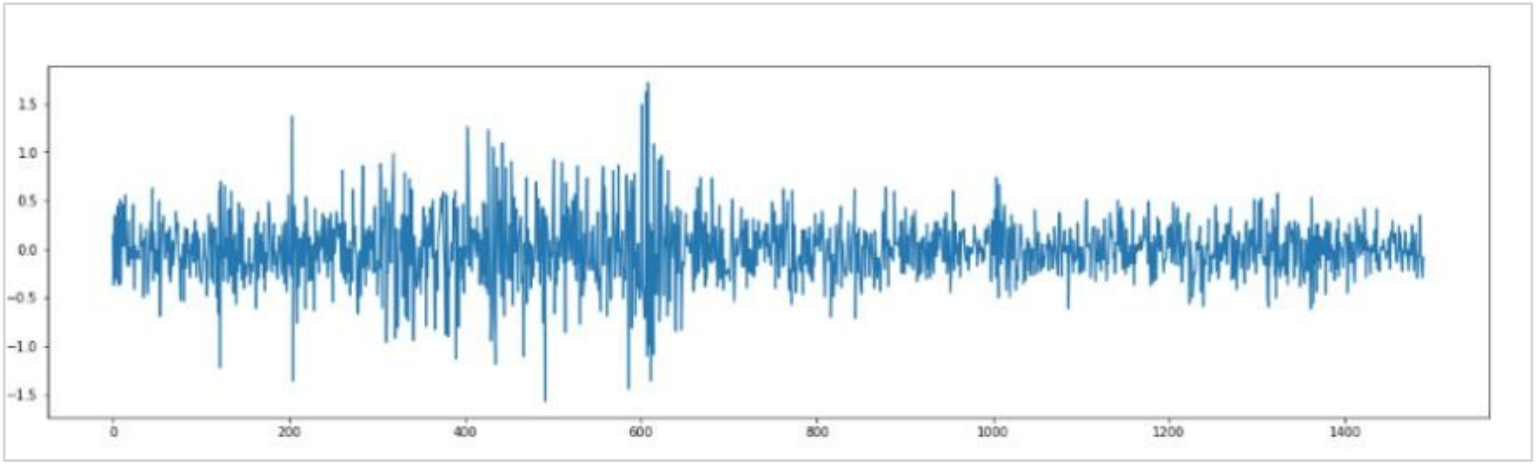
· 스프레드가 고정 비율 때(47쪽)보다 평균으로 더 빨리 회귀하는가?

* Source: http://www.thealgoengineer.com/2014/online_linear_regression_kalman_filter/

칼만 스프레드는 평균으로 더 빨리 회귀한다

시간에 따른 헤지 비율과 스프레드: 고정 비율보다 평균회귀가 빠르다





06

6부

예제: 변동성지수(VIX)와 KOSPI

시장 변수를 평활 성분과 평균회귀 성분으로 분해

시계열 = 랜덤워크 추세 + 1차 자기회귀(AR(1)) 잡음

관측 변수(가격, 신용 스프레드, 변동성)를 평활 성분 X와 평균회귀 성분 Y로 분해한다

$$s_t = X_t + Y_t$$

$$X_t = X_{t-1} + \varepsilon_t, \varepsilon_t \sim N(0, \sigma_\varepsilon^2)$$

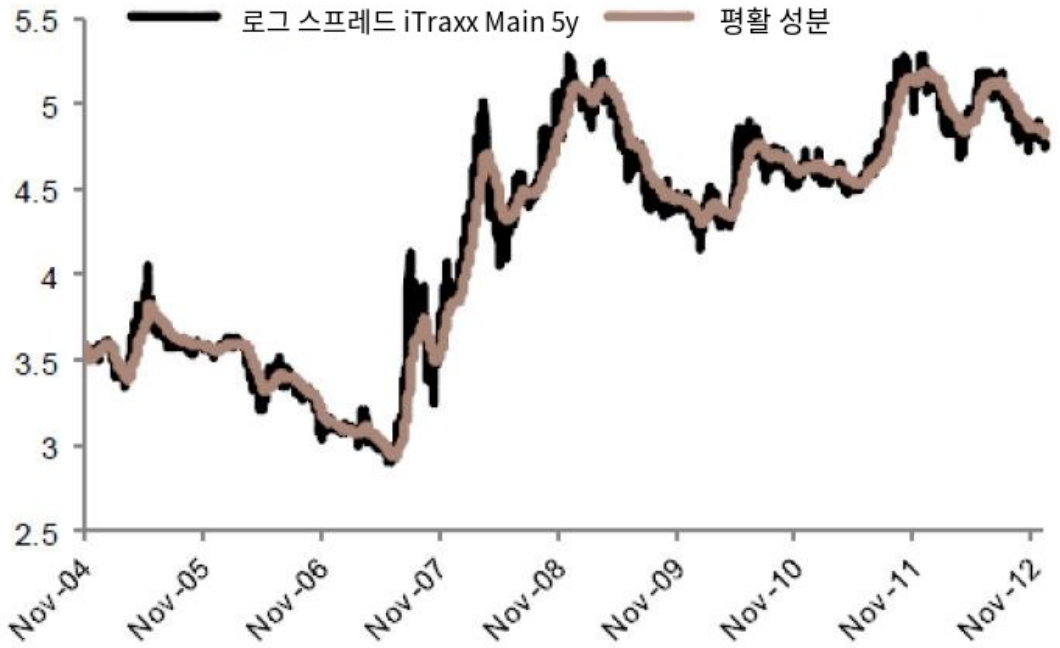
$$Y_t = \kappa Y_{t-1} + \xi_t, 0 < \kappa < 1, \xi_t \sim N(0, \sigma_\xi^2)$$

여기서 s_t는 시점 t의 로그값,
X_t는 랜덤워크 과정, Y_t는 AR(1) 과정이다.

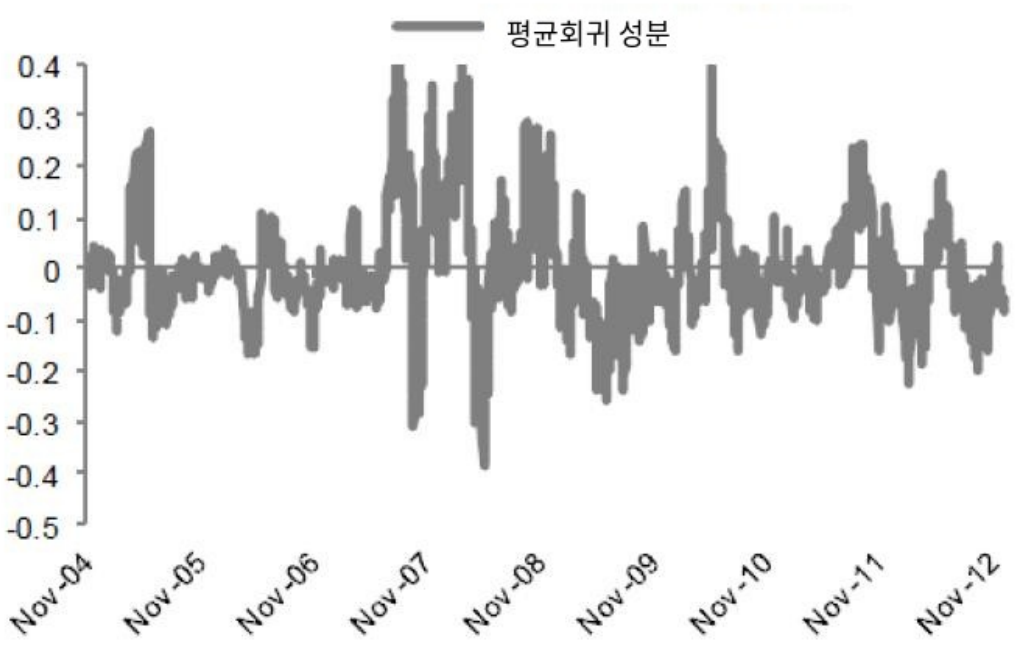
평활 성분은 신용 스프레드 수준을 따라간다

평활 성분과 평균회귀 성분(신용 스프레드, iTraxx Main 5년)

로그 스프레드를 평활 성분과…



…평균회귀 성분으로 분해

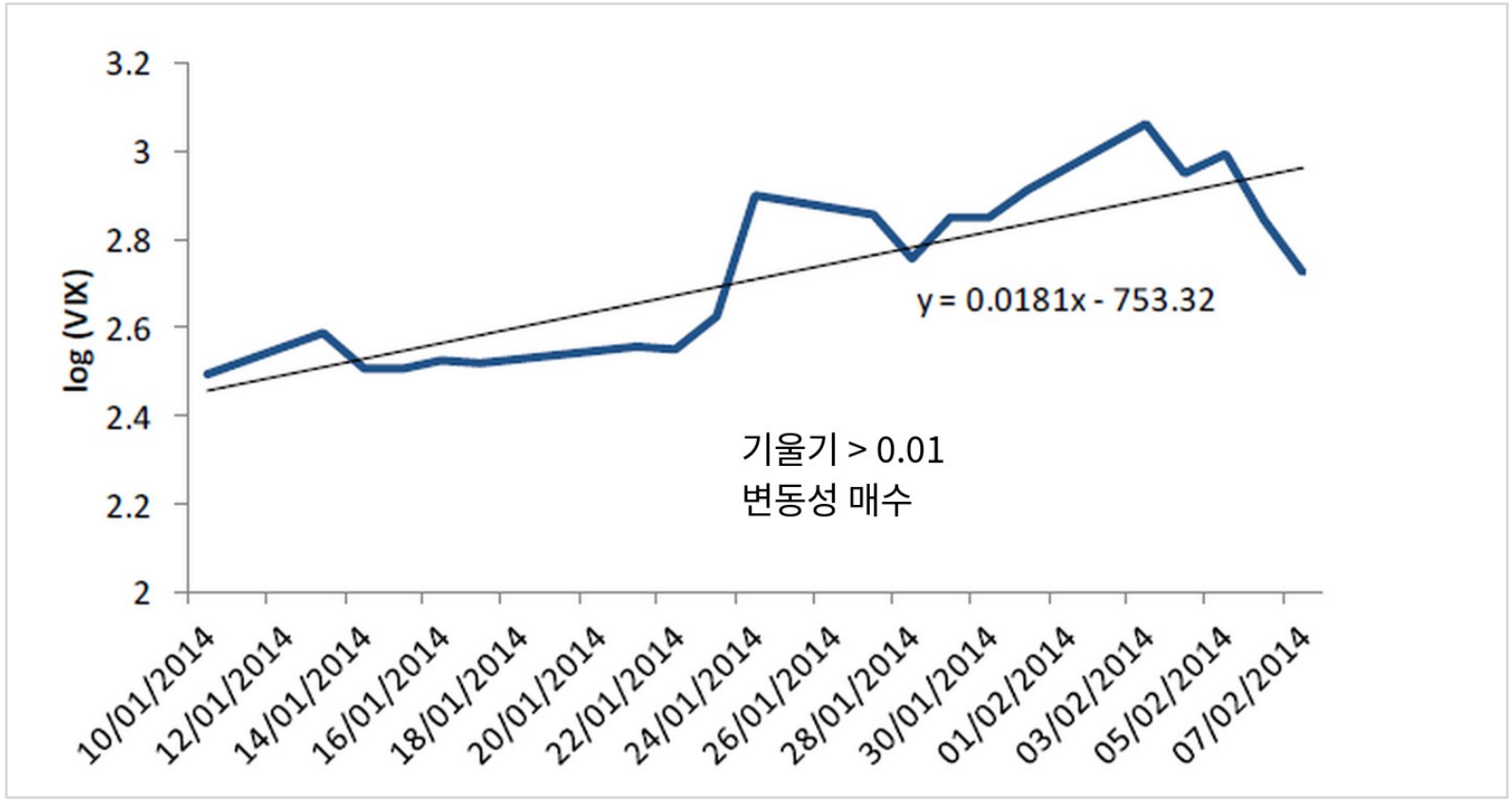


VIX 추세가 오르면 변동성 매수 신호다

평활 성분(VIX)

차트 읽기

- 로그 VIX 평활 성분과 선형 추세
- 기울기 > 0.01 → 변동성 매수

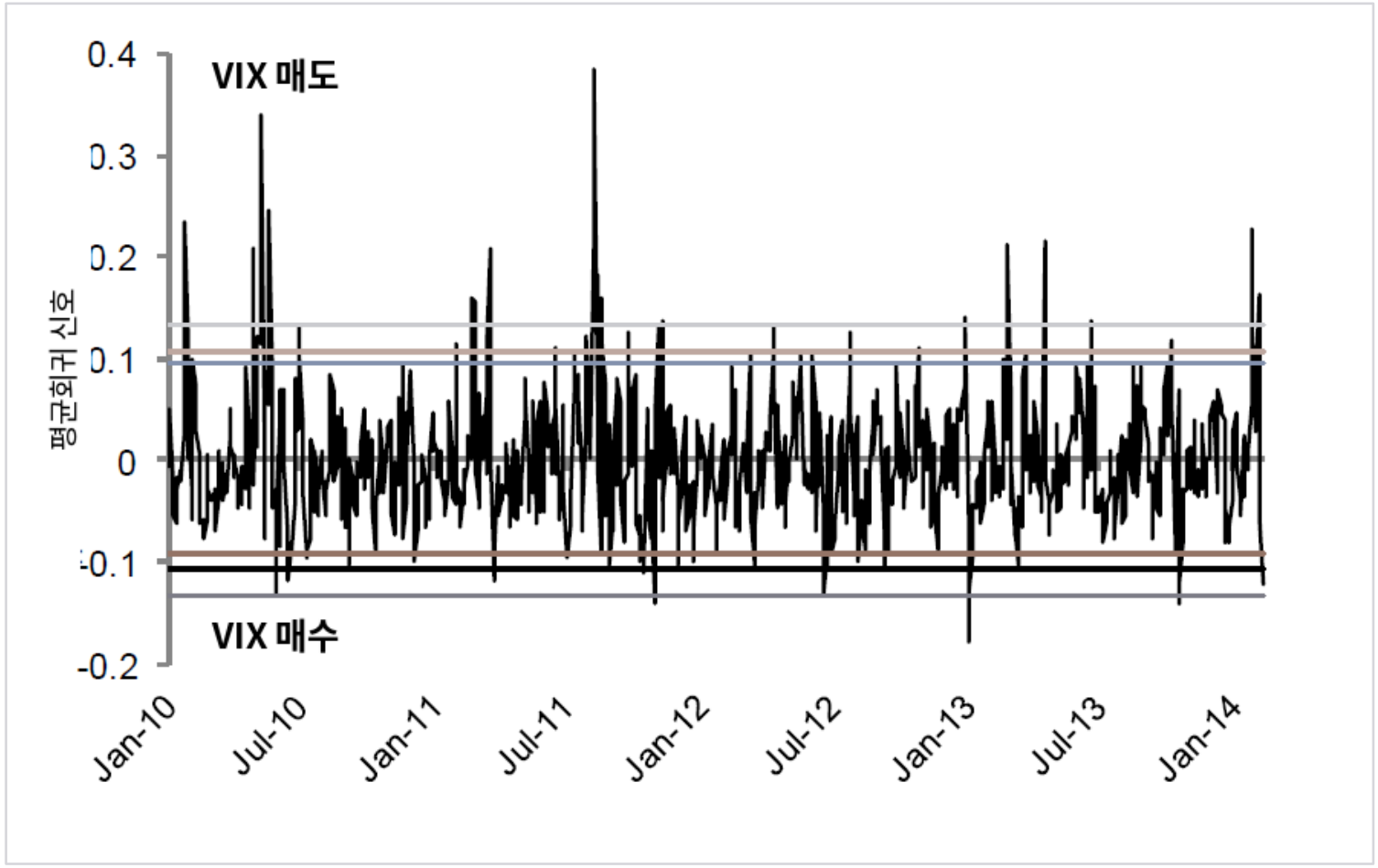


평균회귀 성분의 극단값이 VIX 거래 신호가 된다

평균회귀 성분(VIX)

차트 읽기

- 평균회귀 신호가 높으면 → VIX 매도
- 신호가 낮으면 → VIX 매수



KOSPI에도 같은 분해를 적용한다

KOSPI: 수준, 평활 성분, 평균회귀 성분

네 패널

- KOSPI 수준
- 로그 KOSPI와 평활 성분
- 평균회귀 성분과 매도·매수 구간
- 평균회귀 성분



Kalman Filters 강의가 주요 참고자료다

참고문헌

Kalman Filters

- 지은이: Evgenia “Jenny” Nitishinskaya, Dr. Aidan O'Mahony, Delaney Granizo-Mackenzie.
  알고리즘: David Edwards.